OCLM Phase 2: Adversarial Validation of Canonical Lineage Non-Substitutability

Eighty Symbolic Attack Classes and Eight Health and Reachability Checks for the Oracle–Child Lineage Model

OCLM Phase 2：正統系譜の非置換性に対する敵対的形式検証

Oracle–Child Lineage
Modelに対する80の象徴的攻撃クラスと8つの健全性・到達可能性検証

Hirofumi Kureha (HiRO)

2026-07-21

Contents

Release status

This manuscript is the frozen OCLM Phase 2 public release, version 2.0.0, dated 2026-07-21. Sections 1–11, numerical claims, evidence references, standalone sections, citations, and the preserved formal-evidence package completed the final release audit.

Attribution and Rights

Abstract

The Oracle–Child Lineage Model (OCLM) is a formal model for binding the identity of a Child to its authenticated origin, parent relations, temporal and spatial conditions, generation context, and continuity.

Unlike systems that primarily protect the integrity or ordering of records, OCLM distinguishes between a record that claims a particular history and the canonical lineage through which an entity was actually generated. Its central security objective is **canonical lineage non-substitutability**: an alternative record, identifier, certificate, ledger history, or consensus outcome must not become the canonical past of an existing Child merely by being recorded or accepted as such.

This paper presents Phase 2 of the OCLM formalization program: an adversarial validation campaign against the Phase 1 Tamarin models. The campaign consisted of **80 symbolic adversarial attack classes** and **eight health and reachability properties**. The attack classes covered lineage substitution, origin and parent replacement, time and space mutation, certificate and credential forgery, epoch-key compromise, retroactive rewriting, rollback, revocation bypass, split-view behavior, conflicting transparency roots, evidence splicing, and provenance collisions.

All 80 adversarial properties were resolved without a counterexample trace under the stated models and adversary assumptions. All eight health and reachability properties were reachable, providing evidence that the security results were not obtained from an inert or non-executable model.

The last initially unresolved property, A48, concerned whether the accepted past identity of the same Child could be associated with a different lineage without compromise of a relevant target key. It was decomposed into three helper lemmas concerning Child-ID binding, generated-lineage uniqueness, and the existence of an authenticated generation origin. The three helper lemmas and the final A48 property were verified in the same Tamarin invocation, with all wellformedness checks completing successfully.

The resulting Phase 2 evidence reports:

- 80 of 80 symbolic adversarial attack classes ruled out;
- zero counterexample traces;
- zero unresolved attack classes;
- eight of eight health and reachability properties reachable.

These results are limited to the stated formal models, symbolic cryptographic abstractions, compromise conditions, and attacker capabilities. They do not by themselves establish the security of every physical oracle, implementation, deployment, or cross-chain system. They do, however, provide formal evidence that the modeled OCLM security properties preserve the distinction between an altered record and the canonical lineage of the original Child.

Keywords

Oracle–Child Lineage Model; OCLM; canonical lineage; non-substitutability; formal verification; Tamarin Prover; symbolic protocol analysis; blockchain interoperability; cross-chain identity; provenance; forward security; rollback resistance; revocation; transparency; split-view detection.

1. Introduction

Distributed ledgers, databases, certificate systems, and consensus protocols provide mechanisms for storing, ordering, authenticating, and agreeing upon records [3,4].

These mechanisms are valuable, but they primarily answer questions such as:

- Which record was accepted?
- Which transaction occurred first?
- Which key signed the data?
- Which chain has greater weight?
- Which state is currently recognized by a network?

They do not necessarily answer a different question:

> Does the accepted record possess the same authenticated origin and continuous lineage as the original entity it claims to represent?

This distinction becomes especially important when an entity, asset, credential, or state is represented across multiple systems.

A blockchain may preserve an internally consistent history. Another blockchain may preserve a different internally consistent history. A database may contain an identifier identical to one stored elsewhere. A compromised key may produce a formally valid signature. A majority may accept a replacement record.

None of these facts alone establishes that the replacement belongs to the same canonical lineage as the original entity.

The Oracle–Child Lineage Model addresses this problem by binding Child identity to a structured lineage composed from origin, parent relations, time, space, generation context, and continuity.

Conceptually:

```
Child = G(
   Oracle,
   Parents,
   Time,
   Space,
   Context,
   Continuity
)
```

The resulting lineage is not intended merely as another mutable metadata field. It represents the authenticated generation relationship from which the Child derives its canonical identity.

OCLM therefore separates two concepts:

1. the history currently asserted by a record or system; and
2. the canonical lineage derived from authenticated generation and continuity.

This separation produces the central security property examined in this paper:

> A different lineage must not become the canonical lineage of the original Child merely because it has been recorded, signed, accepted, replicated, or selected by consensus.

1.1 Research Problem

The research problem addressed by OCLM Phase 2 is:

> Can an adversary cause a Child accepted under the OCLM Phase 1 models to be associated with a substituted origin, ancestry, epoch, temporal condition, spatial condition, or continuity relation without satisfying the explicitly modeled compromise conditions?

This question is broader than ordinary record immutability.

A record may remain internally valid while referring to the wrong entity.
A signature may verify while having been produced after key compromise.
A chain may remain internally consistent while diverging from an earlier lineage.
A token may preserve its identifier while losing continuity with its origin.
Two systems may each claim to represent the same asset while preserving incompatible histories.

Accordingly, the Phase 2 validation campaign was designed not simply to re-run the Phase 1 lemmas, but to construct explicit attack properties intended to falsify the lineage guarantees.

The campaign examined whether an attacker could:

- substitute a different lineage under the same Child identity;
- construct accepted certificates without the required origin;
- replace parents, time, space, context, or predecessor relations;
- use later key compromise to rewrite earlier epochs;
- revive revoked states;
- force verifier rollback;
- create conflicting but accepted transparency views;

- splice evidence from unrelated executions;
- or legitimize distinct Children under the same provenance.

1.2 Contributions

This paper makes five principal contributions.

1. A formal adversarial validation battery

It defines and evaluates 80 symbolic adversarial attack classes against the OCLM Phase 1 models.

The battery covers multiple layers of the model rather than testing only one certificate or identity rule.

2. Executability and health validation

It evaluates eight health and reachability properties to demonstrate that the security results are not caused by a model in which relevant protocol executions are impossible.

3. Validation of canonical lineage non-substitutability

It provides formal evidence that, under the stated assumptions, an alternative origin, ancestry, temporal-spatial tuple, or epoch history cannot be accepted as the canonical past of the same Child without satisfying a modeled compromise condition.

4. An integrated proof of the final A48 property

It decomposes the most computationally difficult property into three helper lemmas:

```
H1: Same Child ID→
    Same lineage

H2: Same generated lineage→
    Same public ancestry tuple

H3: Uncompromised acceptance→
    Matching generated origin
```

H1, H2, H3, and the final A48 property were verified in the same Tamarin execution.

5. A higher-layer interpretation for multiple blockchains

It identifies OCLM not as a replacement for blockchains, but as a lineage layer capable of binding records across multiple blockchains, ledgers, and evidence systems.

Individual chains may determine their own internal states. OCLM addresses whether states across those chains belong to the same canonical lineage.

1.3 Main Result

The final validation result is:

```
Symbolic adversarial attack classes: 80
Attack classes ruled out: 80 / 80
Counterexample traces found: 0
Unresolved attack classes: 0

Health and reachability properties: 8
Reachable properties: 8 / 8

A48 helper lemmas: 3
Verified helper lemmas: 3 / 3
```

The 88 principal verification targets were established through:

```
Original warning-free runs: 71
Warning-free direct reruns: 16
Integrated A48 proof: 1
Total: 88
```

The A48 integrated proof additionally verified three helper lemmas in the same invocation.

1.4 Scope of the Claim

The result does **not** claim that OCLM is universally unbreakable.

It does not claim that the 80 attack classes represent every attack possible in all physical and digital environments.

It does not automatically establish the correctness of:

- physical sensors;
- external oracle operators;
- hardware implementations;
- software implementations;
- key-storage devices;
- time and location attestation systems;
- governance processes;
- or cross-chain bridge deployments.

The formal result is narrower and more precise:

> Under the stated OCLM models, symbolic cryptographic abstractions, attacker capabilities, and compromise conditions, no counterexample was found for the 80 tested adversarial properties, and all eight health and reachability properties were reachable.

This distinction is essential.

Phase 2 tests the internal consistency and adversarial resistance of the formalized OCLM properties. Deployment security remains dependent on how real-world evidence is generated, authenticated, transported, and interpreted.

1.5 Paper Organization

The remainder of the paper is organized as follows.

Section 2 defines the conceptual foundations of OCLM and canonical lineage.

Section 3 describes the Phase 1 formal-model foundation inherited by Phase 2.

Section 4 defines the adversary and compromise model.

Section 5 describes the 80 adversarial attack classes and eight health properties.

Section 6 presents the verification methodology and reproducibility environment.

Section 7 reports the complete results.

Section 8 presents the integrated A48 proof.

Section 9 discusses the use of OCLM as a higher layer binding multiple blockchains.

Section 10 states assumptions, limitations, and unresolved implementation questions.

Section 11 concludes the paper.

1.6 Relation to Existing Work

OCLM is built using established symbolic-protocol-analysis methods rather than proposing a new proof engine. The active network-adversary tradition originates in work such as Dolev and Yao's formal treatment of public-key protocols [1], while the Tamarin Prover provides an expressive multiset-rewriting framework for unbounded symbolic analysis of concurrent security protocols [2].

Distributed-ledger research generally focuses on chain-local transaction ordering, consensus, finality, and replicated state. Bitcoin introduced a permissionless peer-to-peer ledger based on proof of work [3], while Tendermint represents a BFT approach to ordering events under adversarial conditions [4]. OCLM does not replace these mechanisms; it asks whether states accepted by one or more such systems preserve the same authenticated generation lineage.

OCLM also intersects with, but is not equivalent to, credential and transparency systems. The W3C Verifiable Credentials Data Model separates issuers, holders, verifiers, claims, and securing mechanisms [6]. Certificate Transparency uses append-only logs and consistency evidence to make misissuance and log equivocation detectable [5]. OCLM draws on related concerns—authenticated evidence, revocation, verifier state, and split views—while making canonical lineage and non-substitutability its primary object.

Cross-chain research has studied atomic exchange [7], cryptocurrency-backed assets and interoperability protocols [8], and the trust assumptions required for communication across distributed ledgers [9]. OCLM's distinct position is to define a higher-order lineage relation: it does not merely coordinate transfer or prove that a chain-local event occurred, but asks whether a destination state is an authenticated continuation of the same origin and predecessor history.

This paper therefore presents OCLM as complementary to symbolic protocol verification, distributed-ledger consensus, credential systems, transparency logs, and cross-chain communication. A complete comparative novelty claim against every provenance or interoperability system is outside the scope of Phase 2 and is not required for the formal result reported here.

2. Conceptual Foundations and Definitions

2.1 Entities, Records, and Evidence

OCLM distinguishes among three different objects:

1. an entity or state that was generated;
2. a record claiming that the entity or state was generated; and
3. evidence supporting the relationship between that record and the generation event.

These objects may correspond, but they are not identical.

A database row is a record.
A blockchain transaction is a record.
A certificate is evidence.
A signature is evidence concerning a key and a message.
A consensus result is evidence that a network accepted a state.

None of these objects, in isolation, is identical to the generation event itself.

OCLM therefore does not define canonical identity solely through the continued existence of a particular record. It defines identity through the authenticated relationship among origin, generation conditions, ancestry, and continuity.

2.2 Oracle

An **Oracle** is a source, authority, mechanism, or evidence boundary that participates in establishing a generation fact.

The term does not necessarily refer to a single trusted server or human administrator.

Depending on the application, an Oracle may be:

- a cryptographic authority;
- a physical sensor;
- a hardware root of trust;
- a legal or administrative authority;
- an external protocol;
- a threshold group;
- an authenticated application;

- a blockchain state;
- or a composition of multiple evidence sources.

The role of the Oracle is not to create truth by declaration.

Its role is to supply authenticated evidence concerning an event, state, condition, or generation relationship.

An Oracle may be compromised, mistaken, unavailable, or contradicted. OCLM therefore treats Oracle evidence as part of a larger lineage composition rather than as an unlimited source of truth.

2.3 Parents and Predecessors

A **Parent** is an entity, state, input, lineage, or authority that participates in the generation of a Child.

A model may contain:

```
Parent A
Parent B
```

or a more general set of parents and predecessors.

Parents need not always represent biological or physical parenthood. They may represent:

- prior ownership states;
- predecessor credentials;
- source and destination systems;
- two parties to an agreement;
- an object and an authorizing authority;
- two source lineages being composed;
- or previous protocol states.

A **predecessor** is the immediately preceding state from which succession is asserted.

Parenthood describes participation in generation.
Predecessorship describes continuity from an earlier state.

The distinction is important because a Child may be generated from several inputs while continuing only one particular prior lineage.

2.4 Child

A **Child** is the entity, state, certificate, asset representation, or derived identity produced through an OCLM-recognized generation event.

A Child is not canonically identified merely because a system assigns it a particular identifier.

Its identity is derived from the lineage under which it was generated.

Conceptually:

```
Child = G(
   Oracle,
   Parents,
   Time,
   Space,
   Context,
   Continuity
)
```

Here, `G` represents an authenticated generation relationship rather than a universal implementation-specific function.

The precise representation differs among the Phase 1 formal theories. Some models include root keys, epoch keys, predecessors, issuers, parent order, time windows, spatial domains, interaction contexts, certificates, revocation state, or transparency state.

The common principle is that the Child derives from a structured lineage rather than from a free-standing label.

2.5 Time

Time represents the temporal conditions under which generation, authorization, observation, succession, issuance, or acceptance occurs.

OCLM does not assume that one timestamp alone proves historical truth.

A temporal binding may include:

- event order;
- validity windows;
- issuance time;
- generation time;
- acceptance time;
- epoch membership;
- expiration;
- revocation order;
- or relations among several events.

The formal models use event-order relations and model-specific temporal fields to distinguish, for example:

- compromise before acceptance;
- compromise after generation;
- revocation before later use;
- and a later epoch compromise from an earlier erased epoch.

This prevents a later event from being automatically interpreted as if it had occurred earlier.

2.6 Space

Space represents the domain in which an event, identity, observation, or authorization is valid.

Space may refer to physical location, but it is not limited to physical geography.

It may represent:

- a physical location;
- a jurisdiction;
- a network;
- a blockchain;
- a computational domain;
- a hardware environment;
- an administrative scope;
- or a protocol namespace.

Two records may contain otherwise similar data while belonging to different spatial or logical domains.

OCLM therefore allows spatial conditions to participate in lineage identity.

A record generated in one domain does not automatically inherit the canonical lineage of a record generated in another domain merely because their visible identifiers match.

2.7 Context

Context is the collection of rules, interactions, environmental conditions, and auxiliary evidence under which generation occurs.

Context may include:

- authorization policy;
- contract state;
- transaction conditions;
- parent interaction;
- execution environment;
- protocol phase;

- governance decision;
- or external evidence.

Context prevents an attacker from extracting individually valid components from unrelated executions and combining them into a new lineage claim.

Evidence that is valid in one interaction does not automatically remain valid when transplanted into another generation context.

2.8 Continuity

Continuity is the authenticated relationship connecting a prior state or lineage to a later state or lineage.

Continuity may be established through:

- a valid predecessor reference;
- an authorized transfer;
- an epoch transition;
- a signed succession certificate;
- a monotonic verifier state;
- revocation-aware transition;
- or a composition contract.

Continuity is not equivalent to visual similarity or identifier reuse.

A later record may have the same name or token number as an earlier record while lacking authenticated continuity with it.

Conversely, a legitimate successor may have a different representation, location, ledger, or implementation while remaining part of the same canonical lineage.

This distinction is central to cross-chain migration.

2.9 Lineage

A **lineage** is the authenticated composition of the elements that determine how a Child arose.

At the conceptual level, it may be represented schematically as:

```
L = Lineage(
   Origin,
   Parents,
   Time,
   Space,
   Context,
   Continuity
)
```

or, in a cryptographically bound implementation:

```
L = H(
   domain_tag,
   Origin,
   Parents,
   Time,
   Space,
   Context,
   Continuity
)
```

This expression is schematic. The Phase 1 theories instantiate the lineage structure with model-specific fields.

In the forward-secure epoch model, for example, lineage is additionally bound to elements such as:

- a root;
- an epoch;
- an epoch public key;

- a predecessor;
- an issuer;
- parent identities;
- temporal fields;
- a validity window;
- a spatial field;
- and interaction context.

The security significance of lineage is that changing a bound component produces a different lineage term under the symbolic model.

2.10 Child ID

A **Child ID** is an identifier derived from or cryptographically bound to a lineage.

Conceptually:

```
ChildID = H(
   child_domain_tag,
   Lineage
)
```

A Child ID is therefore not intended to be an arbitrary label independent of generation history.

Under this construction:

```
same Child ID→
 same bound lineage
```

within the modeled symbolic assumptions.

This relationship was directly tested in the A48 helper lemma H1.

The purpose of the binding is not merely to make collisions computationally difficult. It is to ensure that the identity reference used by the protocol cannot be detached from the lineage that generated it.

2.11 Composition Certificate

A **Composition Certificate** is evidence that a Child or lineage state was generated, composed, issued, or accepted under specified conditions.

A certificate may bind:

- a Child ID;
- lineage;
- origin;
- parents;
- predecessor;
- issuer;
- root authority;
- epoch;
- time and space;
- and relevant context.

A valid signature over a certificate demonstrates that the signing key authenticated the encoded statement under the cryptographic model.

It does not, by itself, prove that the signer was uncompromised or that every external fact encoded in the certificate was physically true.

This issuer–evidence–verifier separation is compatible with, but not identical to, general verifiable-credential models [6].

OCLM therefore evaluates certificate validity together with:

- issuance events;
- generation events;

- compromise order;
- key erasure;
- revocation;
- verifier state;
- and lineage consistency.

2.12 Acceptance

Acceptance is a protocol or verifier event indicating that submitted evidence satisfies the modeled verification conditions.

Acceptance is not defined as an unrestricted declaration of truth.

It means that the evidence passed the rules specified by the model.

The security question is therefore not simply whether acceptance can occur, but:

> Under which preceding events, generation facts, or compromise conditions can acceptance occur?

Many Phase 1 and Phase 2 properties follow the structure:

```
Accepted→
 Legitimate origin or explicitly modeled compromise
```

This makes compromise conditions visible rather than allowing invalid acceptance to be treated as unexplained protocol behavior.

2.13 Canonical Lineage

A **canonical lineage** is the lineage supported by the authenticated origin, parent relationships, temporal-spatial conditions, generation context, and continuity recognized by the OCLM model.

Canonical does not mean:

- most popular;
- longest;
- most replicated;
- most profitable;
- accepted by the largest network;
- or recorded by the most powerful actor.

It means that the lineage preserves the authenticated generation relationship required by the model.

Consensus may select a record.

A canonical-lineage determination asks whether that record belongs to the authenticated succession of the entity it claims to represent.

This produces an important distinction:

```
consensus-selected history≠
 necessarily canonical lineage
```

Consensus may supply evidence.
It does not independently redefine origin.

2.14 Lineage Equivalence

Two records may represent the same canonical lineage even when their serialized forms, storage systems, or network locations differ.

Lineage equivalence depends on whether they preserve the same bound origin and continuity, not whether their raw bytes are identical.

Accordingly:

```
different record≠
 necessarily different lineage
```

and:

```
same visible identifier≠
 necessarily same lineage
```

OCLM evaluates the bound generation relationship rather than superficial record equality.

2.15 Substitution

A **substitution** occurs when a record or entity from a different lineage is asserted to be the canonical continuation or past of an existing Child.

Examples include:

- assigning the same Child ID to a different origin;
- replacing one parent with another;
- altering the relevant time or space;
- inserting an unrelated predecessor;
- combining evidence from separate executions;
- or reproducing an asset representation without authorized continuity.

Substitution attempts to erase the distinction between two different generation histories.

The central OCLM security objective is that substitution must not succeed unless an explicitly modeled compromise condition explains why the verifier can no longer distinguish the evidence.

2.16 Migration

A **migration** is an authorized continuation of a lineage into a different system, ledger, representation, or domain.

Migration may involve:

- movement from Blockchain A to Blockchain B;
- replacement of one credential representation with another;
- rotation from one epoch key to the next;
- transfer between owners;
- transition between protocol versions;
- or relocation into another administrative domain.

Migration does not require every record representation to remain identical.

It requires authenticated continuity.

Conceptually:

```
Original lineage↓
    authorized transition
Successor lineage state
```

The successor may exist on a different blockchain while remaining connected to the same origin.

Migration therefore differs from substitution as follows:

```
Migration:
different representation
+ authenticated continuity
= canonical succession

Substitution:
different lineage
+ missing or fabricated continuity
= non-canonical replacement
```

2.17 Cross-Chain Migration

In a cross-chain setting, the source chain and destination chain may each maintain internally valid records.

OCLM does not determine canonical migration merely by asking whether the destination chain minted a corresponding token or accepted a bridge message.

A canonical cross-chain migration requires evidence connecting:

1. the source lineage state;
2. an authorized transition or lock, burn, transfer, or succession event;
3. the destination generation event;
4. the resulting destination Child;
5. and the continuity between them.

Conceptually:

```
Source Child on Blockchain A↓

Authenticated migration event↓

Successor Child on Blockchain B
```

Without that continuity, a destination-side representation is a new claim, not a canonical successor.

OCLM is therefore intended to bind several blockchain-local histories into one higher-order lineage without requiring the blockchains themselves to become one chain.

2.18 Formal Security Objective

The principal conceptual security objective may be stated as follows.

Let two accepted records claim the same Child identity.

If no relevant root or target epoch key was compromised under the modeled conditions, then the records must correspond to the same canonical lineage and the same bound public ancestry components.

Schematically:

```
Accepted(C1, ChildID)∧
 Accepted(C2, ChildID)∧
 NoRelevantCompromise→
 Lineage(C1) = Lineage(C2)
```

and, for lineage-bound components:

```
Lineage(C1) = Lineage(C2)→
 PublicTuple(C1) = PublicTuple(C2)
```

The public tuple may include model-specific values such as:

- origin;
- epoch;
- epoch public key;
- predecessor;
- issuer;
- root;
- parents;
- time fields;
- validity window;
- and space.

This is the formal basis for canonical lineage non-substitutability.

2.19 Conceptual Summary

OCLM separates identity from mere identifier equality.

It separates historical truth from current record acceptance.

It separates authorized migration from lineage substitution.

It separates key compromise from retroactive modification of generation facts.

Its core conceptual chain is:

```
Authenticated origin↓

Generation conditions↓

Parents and predecessor↓

Time, space, and context↓

Continuity↓

Lineage↓

Child identity
```

A different record may describe this chain.

A different system may store this chain.

A different blockchain may continue this chain.

But a different chain of generation does not become the same canonical lineage merely because it adopts the same visible identity.

3. Phase 1 Formal-Model Foundation

3.1 Inheritance Boundary

Phase 2 is an adversarial validation layer built on top of the OCLM Phase 1 formal models. It does not replace the Phase 1 operational semantics and does not redefine the meaning of Oracle, Parent, Child, generation, acceptance, compromise, revocation, rollback, or transparency.

The frozen Phase 2 evidence contains 17 primary Tamarin theories inherited from Phase 1. For the adversarial battery, a generated theory was created for each attack class by preserving the selected source theory's rules and appending an attack-oriented lemma. The generated theories explicitly mark that the original rules are unchanged.

The ordinary Phase 2 attack query has the following interpretation:

```
exists-trace attack_condition
```

A verified trace would constitute a modeled counterexample. A result of:

```
falsified - no trace found
```

means that the attack condition was not reachable under the selected theory and the stated symbolic assumptions.

A48 required a different proof strategy after direct counterexample search exhausted practical resources. The corresponding all-traces safety property was decomposed into three helper lemmas and then verified together with the final property in one Tamarin invocation. No operational rule was weakened or replaced in the integrated A48 theory; the added material consisted of proof lemmas over the inherited C3 transition system.

3.2 Phase 1 Model Family

The Phase 1 foundation is not a single monolithic theory. It is a sequence of focused models and composition contracts that progressively introduce external authentication, time-space binding, claim collapse, signed composition, forward-secure epochs, monotonic verifier state, revocation, transparency, and generalized multi-epoch composition.

Layer	Tamarin theory	Principal role
Phase 1	`oclm_phase1.spthy`	Foundational Oracle–Parent–Child generation, independent verification, single-use pairing, lineage binding, and observable Oracle equivocation
Phase 1-B	`oclm_phase1b_external.spthy`	External acceptance with signatures, issuance conditions, compromise exceptions, and pair single-use
Phase 1-B1	`oclm_phase1b1_external_signature.spthy`	Minimal external-signature core linking accepted Child identity to legitimate generation or key compromise
Phase 1-B2	`oclm_phase1b2_time_space_window.spthy`	Ordered parent participation inside a time-space window, closure ordering, and at-most-once generation
Phase 1-B3	`oclm_phase1b3_claim_collapse.spthy`	Survival of authenticated Child identity after denial, alternative claims, or malicious consensus claims collapse
Phase 1-C1	`oclm_phase1c1_composition_contract.spthy`	Composition contract joining time-space generation, claim collapse, authentic acceptance, and explicit compromise branches
Phase 1-C2	`oclm_phase1c2_signed_composition_certificate.spthy`	Signed composition certificates and post-collapse verification of certificate-bound lineage
Phase 1-C3	`oclm_phase1c3_forward_secure_epochs.spthy`	Root credentials, epoch keys, key erasure, later compromise, and preservation of past accepted identity
Phase 1-C3A	`oclm_phase1c3a_forward_secure_epoch_core.spthy`	Isolated forward-secure epoch core used to analyze old certificates after later compromise
Phase 1-C3B	`oclm_phase1c3b_c2_c3_composition_contract.spthy`	Composition contract connecting signed C2 generation with C3 forward-secure sealing and acceptance
Phase 1-C4A	`oclm_phase1c4a_verifier_state_monotonicity_core.spthy`	Monotonic verifier epochs and explicit compromise requirements for rollback success

Layer	Tamarin theory	Principal role
Phase 1-C4B	`oclm_phase1c4b_rollback_resistant_composition.spthy`	Rollback-resistant acceptance of composed lineage under current or stale verifier state
Phase 1-C5A	`oclm_phase1c5a_revocation_state_core.spthy`	Revocation lifecycle, missing-update exceptions, verifier rollback, and authority-compromise reactivation
Phase 1-C5B	`oclm_phase1c5b_transparency_split_view_core.spthy`	Transparency checkpoints, forked views, gossip, split-view detection, and equivocation evidence
Phase 1-C5C	`oclm_phase1c5c_revocation_transparency_lineage_composition.spthy`	Contract composing lineage freshness, revocation, and transparency conditions
Phase 1-C6A	`oclm_phase1c6a_multi_epoch_generalized_core.spthy`	Generalized multi-epoch progression, stale-epoch acceptance, and rollback conditions
Phase 1-C6B	`oclm_phase1c6b_generalized_security_composition_contract.spthy`	Highest Phase 1 assume–guarantee contract composing multi-epoch, transparency, revocation, and lineage evidence

The theories are intentionally separated so that a property can be attacked at the layer where its assumptions and events are explicit. This also prevents a large top-level theory from hiding whether a result originates in basic lineage construction, signature validation, epoch erasure, verifier monotonicity, revocation, or transparency.

3.3 Foundational Lineage Generation

The foundational `OCLM_Phase1` theory introduces the minimal lineage relation used throughout the model family.

An Oracle establishes a root context. A complementary pair is issued. Parent A and Parent B participate. A Child is generated from the resulting ancestry and later accepted independently of public claims or node votes.

The foundational safety claims include:

- an accepted Child was previously generated;
- a generated Child has the expected ancestry;
- the complementary pair is consumed at most once;
- the Child's lineage remains bound to its generation tuple; and
- Oracle equivocation becomes observable evidence rather than an alternative Child record automatically acquiring legitimacy.

This layer establishes the first separation between claims and generation facts. Public claims may exist, but they do not participate in the independent `VerifyChild` rule.

3.4 External Authentication, Time–Space Binding, and Claim Collapse

The Phase 1-B family externalizes and strengthens the basic relation.

External authentication

`OCLM_Phase1B_External` and `OCLM_Phase1B1_ExternalSignature` introduce symbolic signatures and explicit key-compromise branches. Their acceptance properties follow the general pattern:

```
Accepted external Child↛
 legitimate generation or modeled compromise
```

This prevents a successful signature check from being interpreted without regard to whether the relevant key was compromised.

Time and space

`OCLM_Phase1B2_TimeSpaceWindow` makes temporal order and spatial scope first-class components of generation. Parent participation must occur within an open window. Generation precedes the matching close event, and one window generates at most one Child.

The model does not claim that an arbitrary timestamp or location string is physically true. It establishes the symbolic ordering and binding conditions that a concrete attestation mechanism would need to supply.

Claim collapse

`OCLM_Phase1B3_ClaimCollapse` models an environment in which Parent A denies, Parent B offers an alternative, or a malicious consensus decision is published. After the claims are collapsed, a legitimately generated and authenticated Child may still be accepted.

The central point is not that claims disappear. It is that contradictory claims do not retroactively replace the authenticated lineage from which the Child arose.

3.5 Composition Contracts and Signed Certificates

Phase 1-C1 joins time-space generation and claim-collapse behavior into a composition contract. The contract exposes both the clean path and explicit compromise paths, allowing proofs to distinguish authentic acceptance from acceptance explained by key compromise.

Phase 1-C2 then adds a signed composition certificate. The certificate binds the generated Child and its ancestry to a signed statement that can be verified after claim collapse.

The C2 properties establish that:

- signed acceptance requires a matching certificate or compromise;
- uncompromised signed acceptance has full ancestry;
- and the signed composition identity is bound unless the modeled key-compromise condition applies.

A certificate is therefore treated as evidence over a structured generation tuple, not as an independent source capable of redefining that tuple.

3.6 Forward-Secure Epochs

Phase 1-C3 introduces a root key, epoch credentials, live epoch keys, epoch-key erasure, later epochs, and explicit compromise events.

The principal intended distinction is temporal:

```
later key compromise≠
 automatic compromise of an erased earlier epoch key
```

The theory models issuance of a composition certificate and erasure of the corresponding epoch key. An attacker may compromise a live later epoch key or the root key under the modeled rules. Verification of an old certificate after later compromise remains tied to the old epoch credential, the generated lineage, and the relevant compromise history.

The canonical C3 theory contains the properties that later became the difficult Phase 2 targets A14, A46, A47, and A48:

- accepted epoch credentials require issuance or root compromise;
- accepted certificates with issued credentials require origin or target-epoch compromise;
- uncompromised acceptance under an erased epoch has full ancestry; and

- past-epoch identity is bound unless a relevant target epoch key or root key was compromised.

C3A isolates the epoch mechanism in a smaller core. C3B composes the signed C2 generation contract with C3 forward-secure sealing and later acceptance.

3.7 Monotonic Verifier State and Rollback Resistance

A lineage verifier may itself retain a recorded epoch or security state. If that state can be silently rolled back, a stale but once-valid record may be accepted as current.

Phase 1-C4A models verifier states progressing from `e0` to `e1` to `e2`. Rollback attempts do not lower the state unless a verifier compromise has occurred under the model.

Phase 1-C4B composes that monotonicity contract with lineage acceptance. It distinguishes:

- acceptance at the latest verifier state;
- attempted acceptance at a stale state;
- and stale acceptance explained by prior verifier compromise.

This layer makes continuity dependent not only on the certificate's history, but also on the verifier's authenticated state progression.

3.8 Revocation, Transparency, and Split Views

Phase 1-C5 addresses two classes of evidence that become important after issuance.

Revocation

C5A models active acceptance, revocation, delayed update observation, current-state rejection, verifier compromise, rollback, and authority-compromise reactivation.

It separates a temporary missing-update exception from an illegitimate reactivation. A revoked certificate does not become active again without the modeled authority-compromise condition.

Transparency and split views

C5B models transparency checkpoints and gossip between views. A compromised log may issue a conflicting checkpoint, but the fork is connected to a prior log compromise, and gossip of incompatible views produces equivocation evidence.

The model therefore does not assume that a transparency log can never fork. It requires that an accepted fork or conflicting root be attributable to the modeled compromise and be detectable through evidence.

Combined contract

C5C composes revocation, transparency, and lineage freshness. A clean acceptance must carry current component evidence. Revoked, forked, or stale acceptance must be explained by a specific exception or prior compromise rather than being silently treated as canonical.

3.9 Generalized Multi-Epoch Composition

Phase 1-C6A generalizes the recorded epoch progression beyond a fixed small sequence. It models advancement to fresh epochs, clean current-epoch acceptance, verifier compromise, rollback by one recorded epoch, and stale acceptance after compromise.

Phase 1-C6B imports the C5C and C6A contracts into a highest-level generalized security composition theory. It covers:

- clean multi-epoch acceptance;
- forked transparency acceptance;
- stale-epoch acceptance;
- revocation exceptions;
- generalized verifier compromise;

- conflicting epoch histories; and
- conflicting transparency roots.

The resulting contract expresses the top-level guarantees expected when lineage, epoch progression, revocation, and transparency evidence are jointly required.

3.10 Composition Semantics and Current Proof Boundary

The Phase 1 model family combines two proof styles.

First, several theories model concrete transition rules directly, including generation, signing, key erasure, compromise, rollback, revocation, and gossip.

Second, the later composition theories import previously established component guarantees as abstract contracts. C6B is therefore an assume–guarantee security composition model. It states and verifies the top-level behavior obtained when the imported C5C and C6A contracts hold.

The current artifact set does **not** claim a single machine-checked refinement proof that automatically derives every C6B transition and guarantee from the complete operational semantics of all lower-level theories in one end-to-end proof object.

This boundary is important:

- Phase 1 verifies each primary theory and its stated composition contracts;
- Phase 2 attacks those theories and contracts as formalized;
- a future refinement phase may mechanically connect the lower-level implementations to the highest-level composition contract more directly.

This limitation does not create a Phase 2 counterexample. It defines the present scope of the formal assurance claim.

3.11 Phase 2 Use of the Foundation

The Phase 2 battery distributes 80 attack classes across the Phase 1 model family. Each attack is placed against the theory that exposes the relevant events and compromise conditions.

Examples include:

- foundational generation and lineage collision attacks against Phase 1;
- external acceptance and provenance attacks against the Phase 1-B family;
- time-space mutation attacks against B2;
- claim-collapse attacks against B3, C1, and C2;
- epoch issuance, erasure, later compromise, and past-identity attacks against C3 and C3A;
- composed forward-security attacks against C3B;
- rollback attacks against C4A and C4B;
- revocation and reactivation attacks against C5A;
- transparency and split-view attacks against C5B;
- combined stale, revoked, and forked acceptance attacks against C5C; and
- generalized multi-epoch and composition attacks against C6A and C6B.

Eight health and reachability properties are evaluated alongside the attack classes. These traces demonstrate that the relevant honest, compromised, revoked, stale, forked, and multi-epoch paths are executable where the model intends them to be executable.

The Phase 2 result must therefore be read as a model-family result rather than as a claim about one isolated lemma. The campaign attacks the lineage concept at several abstraction levels and across multiple state-evolution mechanisms.

3.12 Section Summary

The Phase 1 foundation develops OCLM from basic Child generation into a generalized security composition contract.

Its progression can be summarized as:

```
Generation and ancestry↓

External authentication↓

Time and space↓

Claim collapse↓

Signed composition↓

Forward-secure epochs↓

Monotonic verifier state↓

Revocation and transparency↓

Generalized multi-epoch composition
```

Phase 2 inherits this foundation without weakening its operational rules and asks whether explicitly constructed attack conditions can violate the corresponding lineage guarantees.

4. Adversary and Compromise Model

4.1 Purpose of the Threat Model

Phase 2 does not define one narrative attacker with a single preferred exploit path. It defines a family of symbolic adversarial objectives against the Phase 1 transition systems.

Each attack class asks whether the inherited rules admit a trace in which a prohibited state is reached, such as:

- an accepted Child without the required generation history;
- the same Child identity bound to different lineages;
- a stale, revoked, or forked state accepted without its required exception;
- a historical epoch rewritten through an unrelated later compromise; or
- evidence from separate executions combined into one accepted ancestry.

The threat model is therefore property-oriented. The adversary succeeds whenever the attack condition becomes reachable under the selected theory. The adversary fails for that class when no such symbolic trace exists.

This structure separates the security claim from assumptions about an attacker's motivation. A lineage-substitution trace is a counterexample whether it is produced for economic profit, governance capture, fraud, censorship, bridge manipulation, or another purpose.

4.2 Symbolic Cryptographic Setting

The inherited theories use Tamarin's symbolic term model, within the broader active-adversary tradition associated with Dolev–Yao protocol analysis [1,2]. Depending on the theory, the declared cryptographic built-ins are:

```
hashing
```

or:

```
hashing, signing
```

Cryptographic operations are treated according to their symbolic equations rather than through bit-level computational analysis. In particular, the model does not grant the adversary an unmodeled ability to:

- invert a hash;
- obtain a signing key from a public verification key;
- produce a valid signature without the corresponding signing key;
- or create an equality between distinct free terms merely by assertion.

Fresh values remain unavailable to the adversary unless a rule releases them or a modeled compromise exposes them. Where protocol data is sent through public `Out(...)` facts or received through `In(...)` facts, it is subject to the ordinary symbolic network-attacker treatment of the theory.

This is a perfect-cryptography abstraction. Computational reductions, concrete algorithm parameters, entropy failures, implementation defects, side channels, and cryptanalytic breakthroughs are outside the Phase 2 proof claim.

4.3 Adversarial Knowledge and Message Manipulation

Within the symbolic setting, the attacker may use terms that become available through public outputs, attacker-controlled inputs, or explicit compromise events. Known terms may be replayed, reordered, combined, and presented in later protocol interactions where the rules permit them.

This ability is important for the following attack families:

- replay and single-use violations;
- cross-instance ancestry splicing;
- certificate or credential reuse;
- stale-state presentation;
- conflicting public claims;
- and provenance collisions.

The attacker is not restricted to forwarding complete honest transcripts. It may attempt to combine known components from different executions. The defense therefore cannot rely merely on keeping messages grouped by convention. It must arise from term binding, equality constraints, event ordering, persistent or linear state, and the acceptance rules themselves.

The model does not assume that every network message is delivered, delivered once, or delivered in order. Availability and denial-of-service resistance are not established by the attack battery. The eight health properties show reachability of selected intended traces, not universal liveness under an actively blocking network.

4.4 Explicit Compromise Rather Than Implicit Omnipotence

Privileged failures are represented by explicit state transitions and trace events. A result may therefore distinguish an unexplained security failure from one that is attributable to a modeled compromise.

Across the Phase 1 model family, compromise roles include:

Compromised component	Modeled consequence
Oracle or Oracle signing key	May explain otherwise unauthenticated external or post-collapse acceptance under the affected theory
Root key	May explain forged or unauthorized epoch credentials and root-authorized evidence after the compromise boundary
Live or target epoch key	May explain certificate evidence attributable to that epoch while the modeled key is available or explicitly compromised
Later epoch key	May affect the later epoch but does not automatically expose an erased earlier target epoch key
Composition root	May authorize the specific compromised composition path modeled by the contract
Lineage or epoch verifier	May permit stale-state acceptance or rollback under the corresponding contract
Revocation verifier	May explain acceptance after a compromised revocation-state rollback
Revocation authority	May explain explicit certificate reactivation after compromise

Compromised component	Modeled consequence
Transparency log	May explain conflicting checkpoints, roots, or forked views after log compromise
Generalized composition verifiers	May explain the corresponding stale, revoked, forked, or conflicting branch in the C6 contracts

The table describes modeled consequences, not an unrestricted assertion that compromise of one role grants control over every OCLM component. Compromise is scoped by the rules of the selected theory.

4.5 Time-Indexed Compromise

A central feature of the threat model is that compromise has temporal position.

Security properties distinguish among:

- compromise before generation;
- compromise before issuance;
- compromise before acceptance;
- compromise after acceptance;
- compromise of the target epoch;
- compromise of a different later epoch;
- and compromise of a root or verifier authority.

This distinction prevents a later compromise from being treated as an automatic explanation for every earlier event.

Schematically:

```
Compromise(K) after historical acceptance≠
 Compromise(K) before historical acceptance
```

The exact property determines which ordering is relevant. For example, an acceptance property may permit a compromise exception only when the corresponding compromise event precedes the acceptance event.

The Phase 2 results therefore do not claim that compromised keys remain trustworthy. They claim that the formal consequences of compromise remain bounded by the modeled key, epoch, authority, and event order.

4.6 Forward-Secure Epoch Boundary

The C3 and C3A theories distinguish a live epoch key from an erased epoch key and distinguish the target historical epoch from a later epoch.

A certificate may be issued and the corresponding epoch key erased. A later epoch key may subsequently be compromised. The threat model asks whether this later compromise permits the attacker to reconstruct or substitute the accepted identity of the erased past epoch.

The intended boundary is:

```
compromise of a later epoch≠
 compromise of the erased target epoch
```

The attacker may use an explicit target-old-epoch or root-compromise path when such a path exists in the theory. It may not silently reinterpret later compromise as if it had occurred against the historical target key.

This boundary is exercised by the attack classes concerning:

- post-erasure compromise;
- unauthorized epoch credentials;
- old-certificate acceptance;

- complete ancestry of uncompromised erased-epoch acceptance;
- and past-epoch identity substitution.

4.7 Oracle and External-Evidence Boundary

OCLM uses Oracle evidence as an input to lineage formation, but Phase 2 does not prove that an external physical observation is objectively correct.

The formal models can establish relationships such as:

- the evidence was issued under a modeled Oracle authority;
- a signature verifies under a corresponding key;
- the acceptance event has a matching generation event;
- or a failure is attributable to Oracle-key compromise.

They do not independently establish that:

- a sensor measured the physical world correctly;
- a legal authority acted honestly;
- a human operator entered truthful data;
- a location attestation was physically unforgeable;
- or a hardware root was free of implementation defects.

A concrete deployment must define how physical or institutional facts cross the Oracle boundary. Phase 2 protects the formal lineage relation after those facts have been represented by the model; it does not eliminate the need to secure the evidence source.

4.8 Claims, Denial, and Malicious Consensus

The claim-collapse models deliberately include contradictory public behavior. Parent denial, alternative claims, and malicious consensus claims may occur as trace events.

These actions test whether public narrative can overwrite authenticated generation.

The attacker may therefore influence or produce modeled claims. The security boundary is that claims and consensus events do not, by themselves, modify the generation facts consumed by independent verification.

The threat model does not state that consensus systems are irrelevant. Consensus may determine which record a network presents or stores. OCLM tests the separate question of whether that record retains the authenticated lineage required by the Child identity.

4.9 Cross-Instance and Evidence-Splicing Attacks

Several attack classes attempt to create one accepted Child from otherwise legitimate components belonging to different executions.

Examples include combining:

- a window from one instance;
- Parent A participation from another;
- Parent B participation from a third;
- an unrelated generation or collapse event;
- a credential from one epoch;
- and a certificate from another lineage.

The symbolic attacker is allowed to reuse known terms where the rules make them available. The attack condition then asks whether acceptance can occur without one coherent matching ancestry.

These attacks are stronger than checking each component independently. They test whether the model binds the components to the same root, pair, window, Child, lineage, epoch, and contextual tuple.

4.10 Replay, Single Use, Staleness, and Rollback

The state-oriented portion of the threat model covers four distinct mechanisms.

Replay

Previously available evidence may be presented again. The question is whether replay can create an additional Child or a second valid transition when the model requires one-time use.

Single use

Linear state and consumption rules represent resources such as complementary issuance pairs or generation windows that must not be reused for multiple distinct Children.

Staleness

A previously valid certificate, epoch, lineage state, or verifier state may be presented after a newer state has been established.

Rollback

The attacker may attempt to lower the verifier's recorded state. The models distinguish an attempted rollback from a rollback that succeeds only after an explicit verifier compromise.

A stale record is not necessarily fabricated. Its danger is that authentic old evidence may be represented as current. OCLM therefore binds acceptance to the verifier's authenticated state progression and to explicit compromise exceptions.

4.11 Revocation and Reactivation

The revocation threat model distinguishes among:

- an active certificate;
- a certificate that has been revoked;
- a verifier that has not yet observed the update;
- a current verifier that must reject the revoked certificate;
- a verifier compromised and rolled back;
- and an authority compromised before explicit reactivation.

A missing update is modeled as a specific exception rather than as evidence that revocation never occurred. Likewise, reactivation is not treated as an ordinary state transition unless the theory provides the corresponding authority-compromise path.

The adversarial objective is to make revoked evidence appear canonically active without the required exception or compromise history.

4.12 Transparency, Forks, and Split Views

The transparency models do not assume that a log is incapable of equivocation.

A compromised log may issue a conflicting checkpoint or forked view under explicit rules. The threat model then asks:

- whether a checkpoint can be accepted without matching issuance;
- whether conflicting roots can exist without prior log compromise;
- whether the same certificate can be accepted under incompatible histories;
- whether conflicting gossip produces equivocation evidence;
- and whether a final composed acceptance can use a forked root without the corresponding compromise history.

Thus, fork existence and fork legitimacy are separated. A compromised component may generate a fork, while the model still requires the fork to be attributable and detectable under the stated trace conditions.

4.13 Generalized Composition Adversary

The C6 models expose attack paths at the generalized composition layer. The adversary may attempt to combine:

- stale or conflicting epochs;
- verifier-compromise state;
- revoked-certificate exceptions;
- forked transparency evidence;
- lineage freshness evidence;
- and imported component contracts.

The C6B result is conditional on its assume–guarantee component contracts. Phase 2 attacks the top-level contract as formalized. It does not convert C6B into an end-to-end refinement proof from every lower-level operational rule.

This distinction limits the assurance claim to the abstract composition boundary identified in Section 3.10.

4.14 Attack-Query Semantics

For 79 of the 80 attack classes, the generated theory appends an `exists-trace` lemma describing the prohibited condition.

The interpretation is:

```
exists-trace lemma verified→
 a modeled counterexample trace exists
```

and:

```
exists-trace lemma falsified - no trace found→
 the prohibited condition is unreachable in that theory
```

The Phase 2 term **SURVIVES** means the attack query was falsified with no counterexample trace, or, for A48, that the equivalent all-traces safety property was verified by the integrated proof described below. It does not mean that Tamarin proved every conceivable attack outside the query.

A48 exception

Direct counterexample search for A48 exhausted practical resources. The security statement was therefore expressed as an all-traces property and decomposed into three helper lemmas:

```
H1: same Child ID implies the same lineage
H2: the same generated lineage implies the same public ancestry tuple
H3: uncompromised acceptance has a matching generated origin
```

H1, H2, H3, and the final past-epoch identity property were verified in the same Tamarin invocation with successful wellformedness checks. A48 is counted once among the 80 attack classes; the three helper lemmas are supporting proof obligations rather than additional attack classes.

4.15 Health and Reachability Semantics

Security claims can be vacuous when a model has no executable honest behavior. Phase 2 therefore includes eight `exists-trace` health properties.

For a health property:

```
exists-trace lemma verified→
 the selected intended lifecycle is reachable
```

The eight health properties cover:

- foundational generation and acceptance;
- a complete time-space window;
- honest signed composition;
- old-certificate verification after later compromise;

- rollback-resistant composition;
- revocation lifecycle;
- transparency split-view detection;
- and clean generalized multi-epoch composition.

These results demonstrate selected executability. They do not prove fairness, eventual delivery, termination of every run, or availability under arbitrary denial of service.

4.16 Explicitly Out-of-Scope Capabilities

The Phase 2 adversary does not automatically include capabilities that are absent from the formal rules and symbolic equations. The following remain outside the direct proof claim:

- computational attacks on concrete hash or signature algorithms;
- hash collisions or signature forgeries not represented by compromise;
- random-number-generator failure;
- memory-safety errors, injection, parser ambiguity, and serialization bugs;
- side-channel and fault-injection attacks;
- physical theft or cloning of devices except where abstracted as key compromise;
- false physical observations at the Oracle boundary;
- economic incentives, bribery, governance capture, and legal coercion except where represented by explicit compromise or claim events;
- probabilistic blockchain finality and network-specific reorganization rules;
- denial of service and global liveness;
- implementation-level cross-chain bridge behavior;
- and a complete refinement proof from all lower-level models to C6B.

These exclusions do not weaken the meaning of the verified properties. They define the boundary within which the properties were established.

4.17 Threat-Model Summary

The Phase 2 adversary is strong at symbolic record manipulation and explicit state abuse, but it is not undefined or omnipotent.

It may attempt to:

```
replay known evidence
combine terms from different executions
present stale or conflicting state
publish contradictory claims
trigger modeled compromise paths
exploit revocation and transparency exceptions
and search for any trace satisfying an attack condition
```

It may not silently receive powers that the selected theory does not grant.

The resulting security question is precise:

> Given the inherited Phase 1 rules, symbolic cryptographic equations, public-message behavior, state transitions, and explicit compromise events, is the prohibited lineage or acceptance condition reachable?

Phase 2 answers that question separately for each of the 80 attack classes and verifies executability through eight health properties.

5. Adversarial Validation Battery and Reachability Catalog

5.1 Battery Composition

Phase 2 evaluates 88 principal verification targets: 80 symbolic adversarial attack classes and eight health and reachability properties. The 80 attack classes are not 80 concrete network packets or 80 examples of one exploit. Each is a symbolic property describing a prohibited class of traces against one of the inherited Phase 1 theories.

The final evidence records:

```
Attack properties: 80
Attack classes ruled out: 80 / 80
Counterexample traces found: 0
Unresolved attack classes: 0
Health and reachability checks: 8
Reachable health properties: 8 / 8
```

The complete machine-readable result set is preserved in `catalog/FINAL_88_RESULTS.csv`. The frozen proof logs, generated theories, hashes, and original evidence composition remain inside the included final-evidence archive.

5.2 Result Semantics

For the 79 direct attack-search properties, the generated theory states a prohibited condition as an `exists-trace` lemma.

```
exists-trace attack lemma falsified - no trace found→
 the prohibited trace was not reachable under the stated model
```

A48 was completed through an integrated `all-traces` proof rather than counted as an additional direct attack search. H1, H2, H3, and the final past-epoch identity property were verified in the same Tamarin invocation. A48 remains one of the 80 attack classes; its three helper lemmas are supporting obligations, not additional attacks.

For a health property:

```
exists-trace health lemma verified→
 the intended lifecycle is reachable
```

The health checks prevent a vacuous interpretation in which attack traces are absent only because the protocol cannot execute its intended behavior.

5.3 Attack Taxonomy

Family	Count	Attack IDs
Lineage and Identity Substitution	4	A01, A02, A03, A04
Cross-Instance Ancestry Splicing	4	A05, A06, A07, A08
Replay, Single Use, Staleness, and Rollback	4	A09, A10, A11, A12
Temporal Boundaries of Key Compromise	4	A13, A14, A15, A16
Revocation and Unauthorized Reactivation	5	A17, A18, A19, A20, A71
Transparency and Split-View Acceptance	4	A21, A22, A23, A24
Time, Space, and Oracle Binding	4	A25, A26, A27, A28
Final Composition and Canonical Conflict	4	A29, A30, A31, A32
Lineage, Identity, and Ancestry Invariants	14	A33, A35, A36, A37, A38, A39, A40, A41, A42, A43, A45, A47, A48, A61
Composition and Evidence Boundaries	4	A34, A56, A59, A67
Adversarial Semantics and Resource Reuse	1	A44

Family	Count	Attack IDs
Temporal Boundaries and State Transitions	17	A46, A49, A50, A51, A52, A53, A54, A55, A57, A62, A63, A64, A65, A66, A68, A69, A70
Transparency Forks and Equivocation	4	A58, A60, A72, A73
Provenance Injectivity and Reverse Identity	7	A74, A75, A76, A77, A78, A79, A80

The taxonomy is analytical rather than a claim that each family is disjoint. For example, an epoch rollback property may simultaneously concern temporal compromise boundaries, stale evidence, and canonical identity. Each property is listed under the primary category used by the final evidence set.

5.4 Lineage and Identity Substitution

These properties directly test whether one accepted Child identity can be associated with two different lineages without the compromise condition required by the selected theory.

A01 — SURVIVES
Target model: `oclm_phase1.spthy`
Formal property: `audit_a01_child_lineage_is_bound`
Verification method: original clean run

A02 — SURVIVES
Target model: `oclm_phase1b1_external_signature.spthy`
Formal property: `audit_a02_external_child_identity_is_bound_or_compromised`
Verification method: original clean run

A03 — SURVIVES
Target model: `oclm_phase1c2_signed_composition_certificate.spthy`
Formal property: `audit_a03_signed_composition_identity_is_bound_or_compromised`
Verification method: original clean run

A04 — SURVIVES
Target model: `oclm_phase1c3b_c2_c3_composition_contract.spthy`
Formal property: `audit_a04_forward_secure_composed_identity_is_bound_or_compromised`
Verification method: clean direct rerun

5.5 Cross-Instance Ancestry Splicing

These attacks try to splice otherwise valid parents, windows, generation events, collapse events, or certificates from different protocol instances into one accepted ancestry.

A05 — SURVIVES
Target model: `oclm_phase1.spthy`
Formal property: `audit_a05_generated_child_has_full_ancestry`
Verification method: original clean run

A06 — SURVIVES
Target model: `oclm_phase1c1_composition_contract.spthy`
Formal property: `audit_a06_uncompromised_acceptance_has_full_composed_ancestry`
Verification method: original clean run

A07 — SURVIVES
Target model: `oclm_phase1c2_signed_composition_certificate.spthy`
Formal property: `audit_a07_uncompromised_signed_acceptance_has_full_ancestry`
Verification method: original clean run

A08 — SURVIVES
Target model: `oclm_phase1c3b_c2_c3_composition_contract.spthy`

Formal property: `audit_a08_uncompromised_composed_acceptance_has_full_ancestry`
Verification method: clean direct rerun

5.6 Replay, Single Use, Staleness, and Rollback

These properties test whether consumed issuance resources can be reused, whether a generation window can produce more than one Child, or whether stale verifier state can be accepted without a modeled compromise.

A09 — SURVIVES
Target model: `oclm_phase1.spthy`
Formal property: `audit_a09_complementary_pair_is_single_use`
Verification method: original clean run

A10 — SURVIVES
Target model: `oclm_phase1b2_time_space_window.spthy`
Formal property: `audit_a10_window_generates_at_most_once`
Verification method: original clean run

A11 — SURVIVES
Target model: `oclm_phase1c4a_verifier_state_monotonicity_core.spthy`
Formal property: `audit_a11_rollback_attempt_cannot_lower_state_without_later_compromise`
Verification method: original clean run

A12 — SURVIVES
Target model: `oclm_phase1c4b_rollback_resistant_composition_contract.spthy`
Formal property: `audit_a12_stale_composed_acceptance_requires_prior_verifier_compromise`
Verification method: original clean run

5.7 Temporal Boundaries of Key Compromise

These attacks test whether compromise is incorrectly projected backward in time, whether erased epoch keys can be compromised later, and whether issuance or target-key compromise is required for historical acceptance.

A13 — SURVIVES
Target model: `oclm_phase1c3_forward_secure_epochs.spthy`
Formal property: `audit_a13_erased_epoch_key_cannot_be_compromised_later`
Verification method: clean direct rerun

A14 — SURVIVES
Target model: `oclm_phase1c3_forward_secure_epochs.spthy`
Formal property: `audit_a14_accepted_epoch_credential_requires_issuance_or_root_compromise`
Verification method: clean direct rerun

A15 — SURVIVES
Target model: `oclm_phase1c3a_forward_secure_epoch_core.spthy`
Formal property: `audit_a15_accepted_old_certificate_requires_issuance_or_target_compromise`
Verification method: original clean run

A16 — SURVIVES
Target model: `oclm_phase1c3b_c2_c3_composition_contract.spthy`
Formal property: `audit_a16_composed_acceptance_requires_contract_or_target_compromise`
Verification method: clean direct rerun

5.8 Revocation and Unauthorized Reactivation

These properties test current-state rejection of revoked certificates, unauthorized reactivation, missing-update exceptions, verifier rollback, and final-composition acceptance after revocation.

A17 — SURVIVES
Target model: `oclm_phase1c5a_revocation_state_core.spthy`
Formal property: `audit_a17_certificate_reactivation_requires_prior_authority_compromise`
Verification method: original clean run

A18 — SURVIVES
Target model: `oclm_phase1c5a_revocation_state_core.spthy`
Formal property: `audit_a18_current_verifier_state_never_accepts_revoked_certificate`
Verification method: original clean run

A19 — SURVIVES
Target model: `oclm_phase1c5a_revocation_state_core.spthy`
Formal property: `audit_a19_revoked_acceptance_requires_missing_update_or_verifier_compromise`
Verification method: original clean run

A20 — SURVIVES
Target model: `oclm_phase1c5c_revocation_transparency_lineage_composition_contract.spthy`
Formal property: `audit_a20_revoked_composed_acceptance_requires_explicit_exception`
Verification method: original clean run

A71 — SURVIVES
Target model: `oclm_phase1c6b_generalized_security_composition_contract.spthy`
Formal property: `audit_a71_revoked_generalized_acceptance_requires_explicit_exception`
Verification method: original clean run

5.9 Transparency and Split-View Acceptance

These attacks test checkpoint provenance, conflicting roots, conflicting histories for one certificate, and split-view acceptance without prior log compromise.

A21 — SURVIVES
Target model: `oclm_phase1c5b_transparency_split_view_core.spthy`
Formal property: `audit_a21_accepted_checkpoint_requires_matching_issuance`
Verification method: original clean run

A22 — SURVIVES
Target model: `oclm_phase1c5b_transparency_split_view_core.spthy`
Formal property: `audit_a22_conflicting_checkpoint_roots_require_prior_log_compromise`
Verification method: original clean run

A23 — SURVIVES
Target model: `oclm_phase1c5b_transparency_split_view_core.spthy`
Formal property: `audit_a23_same_certificate_conflicting_transparency_histories_require_compromise`
Verification method: original clean run

A24 — SURVIVES
Target model: `oclm_phase1c5c_revocation_transparency_lineage_composition_contract.spthy`
Formal property: `audit_a24_conflicting_composed_roots_require_prior_log_compromise`
Verification method: original clean run

5.10 Time, Space, and Oracle Binding

These properties test whether Child generation and acceptance remain bound to a matching Oracle issuance, time-space window, parent participation, and closed generation event.

A25 — SURVIVES
Target model: `oclm_phase1b_external.spthy`
Formal property: `audit_a25_external_acceptance_requires_issue_or_compromise`
Verification method: clean direct rerun

A26 — SURVIVES
Target model: `oclm_phase1b2_time_space_window.spthy`
Formal property: `audit_a26_generated_child_has_open_window_ancestry`
Verification method: original clean run

A27 — SURVIVES
Target model: `oclm_phase1c1_composition_contract.spthy`
Formal property: `audit_a27_composed_acceptance_requires_closed_generation_or_compromise`
Verification method: original clean run

A28 — SURVIVES
Target model: `oclm_phase1c2_signed_composition_certificate.spthy`
Formal property: `audit_a28_signed_acceptance_requires_certificate_or_compromise`
Verification method: original clean run

5.11 Final Composition and Canonical Conflict

These attacks target the composed contracts and generalized model, asking whether final acceptance can occur without all component contracts or whether conflicting epoch histories can be accepted without compromise.

A29 — SURVIVES
Target model: `oclm_phase1c5c_revocation_transparency_lineage_composition_contract.spthy`
Formal property: `audit_a29_composed_acceptance_requires_all_verified_component_contracts`
Verification method: original clean run

A30 — SURVIVES
Target model: `oclm_phase1c6a_multi_epoch_generalization_core.spthy`
Formal property: `audit_a30_accepted_epoch_requires_matching_prior_issuance`
Verification method: original clean run

A31 — SURVIVES
Target model: `oclm_phase1c6b_generalized_security_composition_contract.spthy`
Formal property: `audit_a31_generalized_acceptance_requires_c5c_and_c6a_contracts`
Verification method: original clean run

A32 — SURVIVES
Target model: `oclm_phase1c6b_generalized_security_composition_contract.spthy`
Formal property: `audit_a32_conflicting_generalized_epoch_histories_require_prior_compromise`
Verification method: original clean run

5.12 Lineage, Identity, and Ancestry Invariants

This family stresses the foundational ancestry and identity invariants across the model progression, including generation-before-acceptance, parent ordering, claim collapse, external evidence, forward-secure ancestry, and A48.

A33 — SURVIVES
Target model: `oclm_phase1.spthy`
Formal property: `audit_a33_accepted_child_was_generated`
Verification method: original clean run

A35 — SURVIVES
Target model: `oclm_phase1b1_external_signature.spthy`
Formal property: `audit_a35_external_child_requires_generation_or_compromise`
Verification method: original clean run

A36 — SURVIVES
Target model: `oclm_phase1b2_time_space_window.spthy`
Formal property: `audit_a36_generated_child_precedes_matching_close`
Verification method: original clean run

A37 — SURVIVES
Target model: `oclm_phase1b2_time_space_window.spthy`
Formal property: `audit_a37_generated_parent_a_precedes_close`
Verification method: original clean run

A38 — SURVIVES
Target model: `oclm_phase1b2_time_space_window.spthy`
Formal property: `audit_a38_generated_parent_b_precedes_close`
Verification method: original clean run

A39 — SURVIVES
Target model: `oclm_phase1b3_claim_collapse.spthy`

Formal property: `audit_a39_post_collapse_acceptance_requires_generation_or_compromise`
Verification method: original clean run

A40 — SURVIVES
Target model: `oclm_phase1b3_claim_collapse.spthy`
Formal property: `audit_a40_post_collapse_acceptance_has_collapse_ancestry`
Verification method: original clean run

A41 — SURVIVES
Target model: `oclm_phase1b3_claim_collapse.spthy`
Formal property: `audit_a41_post_collapse_child_identity_is_bound_or_compromised`
Verification method: original clean run

A42 — SURVIVES
Target model: `oclm_phase1b_external.spthy`
Formal property: `audit_a42_external_acceptance_requires_generation_or_compromise`
Verification method: clean direct rerun

A43 — SURVIVES
Target model: `oclm_phase1b_external.spthy`
Formal property: `audit_a43_external_child_lineage_is_bound_or_compromised`
Verification method: clean direct rerun

A45 — SURVIVES
Target model: `oclm_phase1c1_composition_contract.spthy`
Formal property: `audit_a45_composed_child_identity_is_bound_or_compromised`
Verification method: original clean run

A47 — SURVIVES
Target model: `oclm_phase1c3_forward_secure_epochs.spthy`
Formal property: `audit_a47_uncompromised_erased_epoch_acceptance_has_full_ancestry`
Verification method: clean direct rerun

A48 — SURVIVES
Target model: `oclm_phase1c3_forward_secure_epochs.spthy`
Formal property: `audit_a48_past_epoch_identity_is_bound_or_target_compromised`
Verification method: integrated all-traces proof with three verified helper lemmas

A61 — SURVIVES
Target model: `oclm_phase1c5c_revocation_transparency_lineage_composition_contract.spthy`
Formal property: `audit_a61_stale_lineage_composed_acceptance_requires_prior_compromise`
Verification method: original clean run

5.13 Composition and Evidence Boundaries

These properties test whether equivocation is evidenced and whether clean composed acceptance retains the current lineage, certificate, transparency, verifier, and generalized security evidence required by the contract.

A34 — SURVIVES
Target model: `oclm_phase1.spthy`
Formal property: `audit_a34_oracle_equivocation_is_evidenced`
Verification method: original clean run

A56 — SURVIVES
Target model: `oclm_phase1c4b_rollback_resistant_composition_contract.spthy`
Formal property: `audit_a56_same_contract_conflicting_verifier_histories_require_compromise`
Verification method: original clean run

A59 — SURVIVES
Target model: `oclm_phase1c5c_revocation_transparency_lineage_composition_contract.spthy`
Formal property: `audit_a59_clean_composed_acceptance_has_current_component_evidence`
Verification method: original clean run

A67 — SURVIVES
Target model: `oclm_phase1c6b_generalized_security_composition_contract.spthy`

Formal property: `audit_a67_clean_generalized_acceptance_has_complete_evidence`
Verification method: original clean run

5.14 Adversarial Semantics and Resource Reuse

This property tests whether the external model accidentally changes the one-time resource semantics inherited from the foundational generation model.

A44 — SURVIVES
Target model: `oclm_phase1b_external.spthy`
Formal property: `audit_a44_external_model_pair_is_single_use`
Verification method: clean direct rerun

5.15 Temporal Boundaries and State Transitions

These properties cover monotonic verifier progression, epoch rollback, stale-epoch acceptance, revocation-state rollback, target-compromise usage, and evidence requirements for current or stale generalized acceptance.

A46 — SURVIVES
Target model: `oclm_phase1c3_forward_secure_epochs.spthy`
Formal property: `audit_a46_accepted_certificate_with_issued_credential_requires_origin_or_epoch_compromise`

Verification method: clean direct rerun

A49 — SURVIVES
Target model: `oclm_phase1c3a_forward_secure_epoch_core.spthy`
Formal property: `audit_a49_erased_old_epoch_key_cannot_be_compromised_later`
Verification method: original clean run

A50 — SURVIVES
Target model: `oclm_phase1c3a_forward_secure_epoch_core.spthy`
Formal property: `audit_a50_accepted_epoch_credential_requires_issuance_or_root_compromise`
Verification method: original clean run

A51 — SURVIVES
Target model: `oclm_phase1c4a_verifier_state_monotonicity_core.spthy`
Formal property: `audit_a51_verifier_state_e1_never_rolls_back_to_e0_without_compromise`
Verification method: original clean run

A52 — SURVIVES
Target model: `oclm_phase1c4a_verifier_state_monotonicity_core.spthy`
Formal property: `audit_a52_verifier_state_e2_never_rolls_back_to_e1_without_compromise`
Verification method: original clean run

A53 — SURVIVES
Target model: `oclm_phase1c4a_verifier_state_monotonicity_core.spthy`
Formal property: `audit_a53_verifier_state_e2_never_rolls_back_to_e0_without_compromise`
Verification method: original clean run

A54 — SURVIVES
Target model: `oclm_phase1c4b_rollback_resistant_composition_contract.spthy`
Formal property: `audit_a54_rollback_resistant_acceptance_requires_c3b_contract`
Verification method: original clean run

A55 — SURVIVES
Target model: `oclm_phase1c4b_rollback_resistant_composition_contract.spthy`
Formal property: `audit_a55_composed_acceptance_uses_latest_state_or_prior_verifier_compromise`
Verification method: original clean run

A57 — SURVIVES
Target model: `oclm_phase1c5a_revocation_state_core.spthy`
Formal property: `audit_a57_verifier_revocation_state_never_rolls_back_without_compromise`
Verification method: original clean run

A62 — SURVIVES
Target model: `oclm_phase1c6a_multi_epoch_generalization_core.spthy`
Formal property: `audit_a62_current_epoch_acceptance_has_current_evidence`
Verification method: original clean run

A63 — SURVIVES
Target model: `oclm_phase1c6a_multi_epoch_generalization_core.spthy`
Formal property: `audit_a63_stale_epoch_acceptance_requires_prior_verifier_compromise`
Verification method: original clean run

A64 — SURVIVES
Target model: `oclm_phase1c6a_multi_epoch_generalization_core.spthy`
Formal property: `audit_a64_generalized_epoch_rollback_requires_prior_verifier_compromise`
Verification method: original clean run

A65 — SURVIVES
Target model: `oclm_phase1c6a_multi_epoch_generalization_core.spthy`
Formal property: `audit_a65_recorded_epoch_transition_never_rolls_back_without_compromise`
Verification method: original clean run

A66 — SURVIVES
Target model: `oclm_phase1c6a_multi_epoch_generalization_core.spthy`
Formal property: `audit_a66_conflicting_direct_epoch_acceptances_require_prior_compromise`
Verification method: original clean run

A68 — SURVIVES
Target model: `oclm_phase1c6b_generalized_security_composition_contract.spthy`
Formal property: `audit_a68_stale_generalized_acceptance_has_stale_epoch_evidence`
Verification method: original clean run

A69 — SURVIVES
Target model: `oclm_phase1c6b_generalized_security_composition_contract.spthy`
Formal property: `audit_a69_stale_generalized_acceptance_uses_epoch_compromise_state`
Verification method: original clean run

A70 — SURVIVES
Target model: `oclm_phase1c6b_generalized_security_composition_contract.spthy`
Formal property: `audit_a70_generalized_epoch_compromise_use_requires_prior_compromise`
Verification method: original clean run

5.16 Transparency Forks and Equivocation

These attacks test whether conflicting gossip produces evidence and whether forked or conflicting final roots can be accepted without a prior transparency-log compromise.

A58 — SURVIVES
Target model: `oclm_phase1c5b_transparency_split_view_core.spthy`
Formal property: `audit_a58_conflicting_gossip_produces_equivocation_evidence`
Verification method: original clean run

A60 — SURVIVES
Target model: `oclm_phase1c5c_revocation_transparency_lineage_composition_contract.spthy`
Formal property: `audit_a60_forked_composed_acceptance_requires_prior_log_compromise`
Verification method: original clean run

A72 — SURVIVES
Target model: `oclm_phase1c6b_generalized_security_composition_contract.spthy`
Formal property: `audit_a72_forked_generalized_acceptance_requires_prior_log_compromise`
Verification method: original clean run

A73 — SURVIVES
Target model: `oclm_phase1c6b_generalized_security_composition_contract.spthy`
Formal property: `audit_a73_conflicting_generalized_transparency_roots_require_prior_compromise`
Verification method: clean direct rerun

5.17 Provenance Injectivity and Reverse Identity

These properties test the reverse direction of identity binding: whether one fully identical provenance tuple can generate or justify two distinct Child IDs without the compromise condition allowed by the theory.

A74 — SURVIVES
Target model: `oclm_phase1.spthy`
Formal property: `audit_a74_same_provenance_distinct_child`
Verification method: original clean run

A75 — SURVIVES
Target model: `oclm_phase1b1_external_signature.spthy`
Formal property: `audit_a75_same_provenance_distinct_child`
Verification method: original clean run

A76 — SURVIVES
Target model: `oclm_phase1b3_claim_collapse.spthy`
Formal property: `audit_a76_same_provenance_distinct_child`
Verification method: original clean run

A77 — SURVIVES
Target model: `oclm_phase1b_external.spthy`
Formal property: `audit_a77_same_provenance_distinct_child`
Verification method: clean direct rerun

A78 — SURVIVES
Target model: `oclm_phase1c1_composition_contract.spthy`
Formal property: `audit_a78_same_provenance_distinct_child`
Verification method: original clean run

A79 — SURVIVES
Target model: `oclm_phase1c2_signed_composition_certificate.spthy`
Formal property: `audit_a79_same_provenance_distinct_child`
Verification method: clean direct rerun

A80 — SURVIVES
Target model: `oclm_phase1c3b_c2_c3_composition_contract.spthy`
Formal property: `audit_a80_same_provenance_distinct_child`
Verification method: clean direct rerun

5.18 Health and Reachability Properties

The eight health properties cover representative honest, compromised, revocation, transparency, and generalized lifecycles. They establish selected reachability, not universal liveness or fairness.

H01 — REACHABLE
Reachable lifecycle: foundational Phase 1 lifecycle is reachable
Model: `oclm_phase1.spthy`
Lemma: `executable`

H02 — REACHABLE
Reachable lifecycle: complete time-space window lifecycle is reachable
Model: `oclm_phase1b2_time_space_window.spthy`
Lemma: `executable_complete_window_lifecycle`

H03 — REACHABLE
Reachable lifecycle: honest signed-composition lifecycle is reachable
Model: `oclm_phase1c2_signed_composition_certificate.spthy`
Lemma: `executable_honest_signed_composition_lifecycle`

H04 — REACHABLE
Reachable lifecycle: valid old-certificate verification after later epoch compromise is reachable
Model: `oclm_phase1c3a_forward_secure_epoch_core.spthy`
Lemma: `executable_old_certificate_after_later_epoch_compromise`

H05 — REACHABLE
Reachable lifecycle: rollback-resistant composition lifecycle is reachable
Model: `oclm_phase1c4b_rollback_resistant_composition_contract.spthy`
Lemma: `executable_rollback_resistant_composed_lifecycle`

H06 — REACHABLE
Reachable lifecycle: revocation lifecycle is reachable
Model: `oclm_phase1c5a_revocation_state_core.spthy`
Lemma: `executable_revocation_lifecycle`

H07 — REACHABLE
Reachable lifecycle: split-view detection lifecycle is reachable
Model: `oclm_phase1c5b_transparency_split_view_core.spthy`
Lemma: `executable_transparency_split_view_lifecycle`

H08 — REACHABLE
Reachable lifecycle: C6B clean multi-epoch lifecycle is reachable
Model: `oclm_phase1c6b_generalized_security_composition_contract.spthy`
Lemma: `executable_generalized_clean_multi_epoch_lifecycle`

5.19 Evidence Composition

The final 88-result set is composed as follows:

Evidence path	Count	Meaning
Original warning-free runs	71	Results that were already clean in the first battery execution
Warning-free direct reruns	16	Properties rerun to remove derivation-check or wellformedness warnings
Integrated A48 proof	1	Final A48 property proved with H1, H2, and H3 in one invocation
Total	**88**	**All principal verification targets**

This composition distinguishes the logical result from the cleanliness of the proof artifact. Intermediate warning-bearing logs are retained as historical source material, but the final result set points to warning-free direct evidence or to the integrated A48 proof.

5.20 Interpretation

The battery did not find a modeled trace in which the tested lineage, ancestry, state-transition, revocation, transparency, or provenance guarantees failed without the compromise or exception specified by the corresponding property. The eight reachable lifecycles show that the tested theories retain representative executable behavior.

These results support the Phase 2 claim within the exact symbolic scope of the models. They do not establish that every implementation, physical Oracle, blockchain bridge, or governance process is secure, and they do not claim that the 80 attack classes exhaust all future attacks.

6. Verification Methodology and Reproducibility

6.1 Evidence Basis

Phase 2 was conducted against the frozen Phase 1 Tamarin source set and was later normalized into a final evidence package containing the models, generated attack theories, proof logs, machine-readable result tables, validation metadata, and SHA-256 manifests.

The frozen Phase 2 evidence archive used by this paper is:

```
oclm-phase2-final-evidence-20260721T050537Z.zip
SHA-256:
de36e384b7583ddde8dfc655ac51eeb517bb64011b5cf4d59a850aad62abb51e
```

The archive passed a ZIP integrity test before inclusion in the publication draft. The evidence files cited in this section are retained inside that archive rather than reconstructed from memory or from later summaries.

6.2 Verification Environment

The initial 88-target battery recorded the following execution environment:

Component	Recorded value
Operating system	macOS 26.5.2, arm64
Darwin kernel	25.5.0
Tamarin Prover	1.12.0
Maude	3.5.1
Graphviz	15.1.0
Python	3.9.6
Initial worker count	4
Initial per-target timeout	1,800 seconds
Source mode	Local directory
Source root	`~/Desktop/oclm`

The Tamarin build recorded in the logs reports a compilation timestamp of 2026-03-08 09:49:47 UTC. The recorded Git revision and branch were `UNKNOWN`; therefore this paper identifies the verifier by its reported version and captured executable output rather than by an unverified source revision.

6.3 Source-Integrity Preflight

Before attack execution, the battery performed a preflight comparison over the 17 Phase 1 Tamarin theories.

For each theory, the preflight compared:

- filename;
- SHA-256 digest;
- byte size;
- rule count; and
- lemma count.

The preflight recorded:

```
expected model files: 17
actual model files: 17
manifest result: PASS
```

All expected and actual entries matched. This matters because Phase 2 was intended to attack the already frozen Phase 1 theories, not an altered or simplified substitute.

The attack-generation process preserved the protocol rules of the selected source theory and introduced a dedicated adversarial property. The generated theory was separately hashed and stored. Phase 2 therefore distinguishes:

1. the canonical Phase 1 source model;
2. the generated attack theory containing the test lemma; and
3. the proof log produced from that generated theory.

6.4 Counterexample-Oriented Attack Construction

Most Phase 1 security statements were universal trace properties. For adversarial validation, their failure conditions were expressed as explicit existential attack lemmas.

Schematically, a source property of the form:

```
All x #i.
   Accepted(x) @ #i
   ==> RequiredCondition(x, #i)
```

was challenged by an attack property of the form:

```
Ex x #i.
   Accepted(x) @ #i
   & not RequiredCondition(x, #i)
```

The generated attack lemma was declared as an `exists-trace` property. This permits the verifier to search directly for a trace witnessing the proposed violation.

Not every attack was a literal negation of one pre-existing lemma. Some Phase 2 properties, including the provenance-injectivity tests, were newly written adversarial formulations. In every case, however, the target was a concrete symbolic counterexample condition rather than an informal claim that a model "looked secure."

6.5 Verdict Semantics

The verdict rules were intentionally asymmetric for attack and health properties.

6.5.1 Attack properties

For an `exists-trace` attack lemma:

Tamarin result	Phase 2 interpretation
`verified`	A counterexample trace exists; the attack is **BROKEN**
`falsified - no trace found`	No modeled attack trace was found; the attack **SURVIVES**
timeout, incomplete analysis, or invalid artifact	**UNVERIFIED**

Thus, "SURVIVES" does not mean that a universal theorem was inferred merely from silence. It means that Tamarin completed the specified symbolic search and falsified the existential attack property under the model.

6.5.2 Health and reachability properties

For an `exists-trace` health lemma:

Tamarin result	Phase 2 interpretation
`verified`	The selected lifecycle is **REACHABLE**
`falsified`	The selected lifecycle is not demonstrated reachable
timeout or invalid artifact	**UNVERIFIED**

The eight health properties are representative executability checks. They do not prove universal liveness, fairness, or availability.

6.6 Clean-Artifact Criteria

The first battery established the logical direction of most results, but publication required a stricter evidence standard. A result was treated as a clean final artifact only when all of the following were present:

1. the expected target verdict line;
2. `All wellformedness checks were successful.`;
3. no derivation-check timeout marker;
4. no non-zero wellformedness warning count;
5. the expected model and log files; and
6. SHA-256 digests recorded in the normalized result set.

This distinction separates two questions:

- Did the symbolic attack search find a counterexample?
- Is the preserved proof artifact clean enough to support publication and independent review?

Intermediate logs containing a completed search result but a derivation-check or wellformedness warning were retained as historical evidence, but they were not used as the final direct evidence when a warning-free rerun was available.

6.7 Initial Battery Execution

The initial battery contained 88 principal targets:

```
80 adversarial attack properties
 8 health and reachability properties
88 total targets
```

The recorded battery interval was:

```
started: 2026-07-20T08:06:00.176168Z
completed: 2026-07-20T08:39:01.600707Z
```

The orchestration layer used four concurrent jobs and a per-target timeout of 1,800 seconds. The ordinary per-property command shape was:

```
tamarin-prover <model-or-generated-theory.spthy> \
  --prove=<target-lemma>
```

The initial aggregate result was:

```
attack SURVIVES: 76
attack BROKEN: 0
attack UNVERIFIED: 4
health REACHABLE: 8
health failed: 0
```

The four initially unresolved attacks were A14, A46, A47, and A48. Their unresolved state was treated as a computational result, not as a security failure and not as a successful proof.

6.8 Warning-Free Direct Reruns

Sixteen principal targets were later replaced by warning-free direct reruns. This set consisted of fifteen attack properties and one health property.

The reruns used the same stored theories and target lemmas, while enabling source saturation and a search heuristic that proved effective for the larger C3 search spaces. The general command form was:

```
tamarin-prover <theory.spthy> \
  --derivcheck-timeout=<seconds> \
  --auto-sources \
  --heuristic=s \
  --prove=<target-lemma> \
  +RTS -N2 -RTS
```

The derivation-check timeout was selected per rerun:

- 600 seconds for the grouped cleanup, H03, and A14;
- 1,200 seconds for A46; and
- 1,800 seconds for A47.

The direct reruns produced clean final evidence for:

```
A04, A08, A13, A14, A16,
A25, A42, A43, A44, A46,
A47, A73, A77, A79, A80,
and H03
```

Every one of those attack properties ended with `falsified - no trace found`; H03 ended with `verified`. All corresponding final logs recorded successful wellformedness checks.

6.9 Integrated A48 Proof

6.9.1 Reason for decomposition

A48 was the only principal target not closed by the ordinary existential attack search. Its direct search explored a substantially larger state space and did not complete within the practical resource budget used for the battery.

Rather than weaken the original C3 protocol rules or reduce the security property, the proof obligation was decomposed into three helper lemmas:

```
H1: same Child ID
   -> same lineage

H2: same generated lineage
   -> same public ancestry tuple

H3: uncompromised acceptance
   -> matching generated origin
```

The final A48 statement then used those structural facts to establish that two accepted past identities for the same Child must agree on the bound tuple unless a modeled target epoch key or root key was compromised before a relevant acceptance.

6.9.2 Integrated invocation

H1, H2, H3, and the final A48 property were selected in the same Tamarin invocation:

```
tamarin-prover \
  oclm_phase1c3_a48_decomposed_h1_h2_h3.spthy \
  --derivcheck-timeout=1800 \
  --auto-sources \
  --heuristic=s \
  --prove=a48_h1_same_child_id_implies_same_lineage \
  --prove=a48_h2_same_generated_lineage_implies_same_public_tuple \
  --prove=a48_h3_uncompromised_acceptance_has_generated_origin \
  --prove=past_epoch_identity_is_bound_or_target_compromised \
  +RTS -N2 -RTS
```

The integrated log records:

```
H1: verified (6 steps)
H2: verified (2 steps)
H3: verified (52 steps)
A48: verified (8 steps)

All wellformedness checks were successful.
Processing time: 707.76 seconds
```

The integrated artifacts are fixed by:

```
model SHA-256:
6a674d76a6c37479a10c727387a9a5771b7557064781ea4411b658e9bf355133

log SHA-256:
7a3b2b0e6b64e2d8bfca323dce4ca49c267fb78286044a11e4c39645a4a714c1
```

The shell wrapper completed the Tamarin run and produced the full log, but a later macOS `awk` post-processing command failed before the already stored process status was printed or written. The final metadata therefore records the exit status as `NOT_CAPTURED_POSTPROCESS` rather than inventing a value. The proof log itself is complete, contains all four verification results and the successful wellformedness statement, and is preserved under the digest above.

6.10 Final Evidence Normalization

The final evidence builder combined three evidence paths:

Evidence path	Principal targets
Original warning-free runs	71
Warning-free direct reruns	16
Integrated A48 proof	1
Total	**88**

The normalization step performed machine checks before writing the final table. It required:

- exactly 88 unique principal IDs;
- exactly 80 attack rows;
- exactly eight health rows;
- `SURVIVES` for every attack row;
- `REACHABLE` for every health row;
- existence of each referenced model and log;
- expected verdict text in each referenced log;
- clean wellformedness evidence for the selected final artifact; and
- SHA-256 calculation over each selected model and log.

A48 was treated specially: the normalized A48 row points to the integrated all-traces proof rather than to the timed-out direct attack search.

The normalization completed with:

```
FINAL VALIDATION PASSED
ATTACK SURVIVES: 80/80
COUNTEREXAMPLES: 0
UNRESOLVED: 0
HEALTH REACHABLE: 8/8
A48 HELPERS VERIFIED: 3/3
```

6.11 Frozen Result Files

The primary normalized result files are:

```
FINAL_88_RESULTS.csv
SHA-256:
4c9451cdb967914940c63742cc15bcb65ac6cf7870973becf8e3e6e942147e26

A48_HELPER_RESULTS.csv
SHA-256:
1de210e2480955b57e287af3dbb0853f69f85a4ad753f763081ad83643770405

FINAL_REPORT.md
SHA-256:
0fc16e9416e0d56c49cd0d13cb92dc5958916cbb43e871b46a9c96d9adc19da8

FINAL_VALIDATION.json
SHA-256:
428b0de447dbf3144b8c0c91fbbed40e6a2bad5b022917a8f729fc38d53391dd

MANIFEST.sha256
SHA-256:
fedfe11b3134780a8cdd425c2eeb5b75acfa2e6e618cd83e531047b111fc3960
```

The final CSV is the authoritative machine-readable map from each principal target to its selected final model, log, verification method, verdict, and evidence digest.

6.12 Reproduction Levels

Independent review can be performed at three levels.

Level 1: Package-integrity verification

This level confirms that the downloaded release is byte-identical to the published archive and that the internal manifest matches.

```
shasum -a 256 oclm-phase2-final-evidence-20260721T050537Z.zip
unzip -tq oclm-phase2-final-evidence-20260721T050537Z.zip
```

After extraction:

```
cd oclm-phase2-final-evidence-20260721T050537Z
shasum -a 256 -c MANIFEST.sha256
```

Level 2: Result-to-log audit

This level inspects `FINAL_88_RESULTS.csv`, opens every referenced evidence log, confirms the expected verdict and clean wellformedness statement, and recalculates model and log hashes.

The publication package includes a verification helper for this artifact-level audit. It does not rely on the original absolute `/Users/hiro/...` paths embedded in historical metadata.

Level 3: Computational rerun

This level reruns selected or all Tamarin targets from the stored theories.

For ordinary attack targets:

```
tamarin-prover models/generated_theories/<attack-theory>.spthy \
  --prove=<attack-lemma>
```

For health targets:

```
tamarin-prover models/phase1_tamarin/<phase1-model>.spthy \
  --prove=<health-lemma>
```

For the final A48 result, the integrated command in Section 6.9.2 should be used.

The frozen final evidence archive preserves the theories, logs, result tables, and metadata needed for per-property reruns and auditing. The original Python orchestration wrapper that launched the initial parallel battery is not part of the frozen final-evidence archive; therefore byte-for-byte reproduction of that wrapper's scheduling behavior is not claimed. The mathematical targets and the Tamarin invocations remain independently rerunnable.

6.13 Reproducibility Boundaries

Several practical factors can change runtime without changing the target property:

- Tamarin and Maude versions;
- processor architecture and available memory;
- source-saturation behavior;
- heuristic choice;
- parallel runtime settings;
- derivation-check timeout; and
- implementation changes in future verifier versions.

A rerun that times out is not a counterexample and is not a proof. A verifier-version difference should be reported together with the exact environment and command used.

Conversely, artifact-level verification does not rerun symbolic search. It establishes that the reviewer is examining the same models, logs, and normalized decisions that form the Phase 2 publication record.

6.14 Methodological Summary

The Phase 2 method can be summarized as follows:

```
freeze Phase 1 source models
      -> verify source manifest
      -> generate explicit attack theories
      -> run 80 attack searches and 8 health checks
      -> retain counterexamples, timeouts, and logs without reinterpretation
      -> rerun warning-bearing completed targets cleanly
      -> decompose and integrate A48 without weakening protocol rules
      -> normalize exactly 88 principal results
      -> hash models, logs, tables, and archive
      -> publish the frozen evidence
```

This process does not convert model assumptions into claims about every implementation. It provides a reproducible chain from a stated symbolic attack property to the exact theory, log, verdict rule, and digest used in the final Phase 2 result.

7. Complete Verification Results

7.1 Final Outcome

The normalized Phase 2 evidence contains exactly 88 principal verification targets: 80 symbolic adversarial attack classes and eight health and reachability properties.

The final outcome is:

```
Symbolic adversarial attack classes: 80
Attack classes ruled out: 80 / 80
Counterexample traces found: 0
Unresolved attack classes: 0

Health and reachability properties: 8
Reachable health properties: 8 / 8
```

For 79 attack classes, the prohibited condition was expressed as an `exists-trace` property and the final selected Tamarin evidence reported `falsified - no trace found`. A48 was closed by an integrated `all-traces` proof in which H1, H2, H3, and the final past-epoch identity property were verified in the same invocation.

The result is a statement about the tested symbolic models and assumptions. It is not an empirical probability of compromise and is not a claim that every attack outside the formalized threat model has been enumerated.

7.2 Evidence Composition

The 88 principal results were selected from three evidence paths.

Evidence path	Count	Composition
Original warning-free runs	71	64 attacks + 7 health properties
Warning-free direct reruns	16	15 attacks + H03
Integrated A48 all-traces proof	1	A48, supported by H1/H2/H3 in the same invocation
Total principal targets	88	80 attacks + 8 health properties

The 16 clean direct reruns replaced warning-bearing completed results with warning-free final evidence. They did not add new principal targets. The three A48 helper lemmas are supporting obligations and are not counted among the 80 attack classes or the 88 principal targets.

7.3 Results by Attack Family

Every attack family in the Phase 2 taxonomy completed with all selected properties in `SURVIVES`, zero counterexamples, and zero unresolved targets.

Attack family	Targets	SURVIVES	Counterexamples	Unresolved
Lineage and Identity Substitution	4	4/4	0	0
Cross-Instance Ancestry Splicing	4	4/4	0	0
Replay, Single Use, Staleness, and Rollback	4	4/4	0	0
Temporal Boundaries of Key Compromise	4	4/4	0	0
Revocation and Unauthorized Reactivation	5	5/5	0	0
Transparency and Split-View Acceptance	4	4/4	0	0
Time, Space, and Oracle Binding	4	4/4	0	0
Final Composition and Canonical Conflict	4	4/4	0	0
Lineage, Identity, and Ancestry Invariants	14	14/14	0	0
Composition and Evidence Boundaries	4	4/4	0	0
Adversarial Semantics and Resource Reuse	1	1/1	0	0
Temporal Boundaries and State Transitions	17	17/17	0	0
Transparency Forks and Equivocation	4	4/4	0	0
Provenance Injectivity and Reverse Identity	7	7/7	0	0

The result is not concentrated in one narrow identity lemma. It spans lineage substitution, ancestry splicing, one-time resource use, temporal compromise boundaries, revocation, rollback, transparency, generalized composition, and provenance injectivity.

7.4 Results Across the Phase 1 Model Set

The final 88 targets cover all 17 Phase 1 Tamarin models. The table below counts the principal attack and health properties associated with each model in `FINAL_88_RESULTS.csv`.

Phase 1 model	Attack targets	Health targets	Total	Selected results
`oclm_phase1c6b_generalized_security_composition_contract.spthy`	9	1	10	PASS
`oclm_phase1.spthy`	6	1	7	PASS
`oclm_phase1b2_time_space_window.spthy`	5	1	6	PASS
`oclm_phase1c5c_revocation_transparency_lineage_composition_contract.spthy`	6	0	6	PASS
`oclm_phase1c6a_multi_epoch_generalization_core.spthy`	6	0	6	PASS
`oclm_phase1b_external.spthy`	5	0	5	PASS
`oclm_phase1c2_signed_composition_certificate.spthy`	4	1	5	PASS
`oclm_phase1c3_forward_secure_epochs.spthy`	5	0	5	PASS
`oclm_phase1c4b_rollback_resistant_composition_contract.spthy`	4	1	5	PASS
`oclm_phase1c5a_revocation_state_core.spthy`	4	1	5	PASS
`oclm_phase1c5b_transparency_split_view_core.spthy`	4	1	5	PASS
`oclm_phase1b3_claim_collapse.spthy`	4	0	4	PASS

Phase 1 model	Attack targets	Health targets	Total	Selected results
oclm_phase1c1_composition_contract.spthy	4	0	4	PASS
oclm_phase1c3a_forward_secure_epoch_core.spthy	3	1	4	PASS
oclm_phase1c3b_c2_c3_composition_contract.spthy	4	0	4	PASS
oclm_phase1c4a_verifier_state_monotonicity_core.spthy	4	0	4	PASS
oclm_phase1b1_external_signature.spthy	3	0	3	PASS

The largest single allocation is the generalized C6B composition contract with ten principal targets. The foundational model contributes seven. The remaining targets are distributed across external signatures, time-space windows, claim collapse, composition contracts, forward-secure epochs, verifier monotonicity, rollback resistance, revocation, transparency, and multi-epoch generalization.

A48 is recorded against the C3 forward-secure epoch model. Its final evidence uses a derived C3 theory that adds helper lemmas without weakening the inherited protocol rules.

7.5 Health and Reachability Results

All eight health properties were reachable.

```
H01 foundational Phase 1 lifecycle REACHABLE
H02 complete Time/Space window lifecycle REACHABLE
H03 honest signed-composition lifecycle REACHABLE
H04 old certificate after later epoch compromise REACHABLE
H05 rollback-resistant composed lifecycle REACHABLE
H06 revocation lifecycle REACHABLE
H07 transparency split-view lifecycle REACHABLE
H08 generalized clean multi-epoch lifecycle REACHABLE
```

These checks do not prove universal liveness, fairness, or deployment availability. They establish that representative honest, compromised, revocation, transparency, and generalized executions remain reachable in the same model set used for the security analysis.

The combined interpretation is therefore:

```
attack trace absent
+ intended representative lifecycle reachable→
 non-vacuous evidence for the tested property
```

7.6 Principal Security Findings

7.6.1 Lineage identity remained bound

A01–A04 tested lineage and identity substitution from the foundational model through external signatures, signed composition, and forward-secure composition. All four survived.

This means that the tested theories did not admit the same accepted Child identity under a different bound lineage without the compromise condition explicitly allowed by the corresponding model.

7.6.2 Cross-instance ancestry splicing was not reachable

A05–A08 attempted to combine otherwise meaningful Oracle, parent, window, generation, collapse, or certificate evidence from different protocol instances. All four survived.

The result supports the requirement that accepted ancestry must be linked within the matching generation instance rather than assembled from unrelated valid fragments.

7.6.3 Later compromise did not automatically rewrite earlier lineage

The key-compromise and temporal-transition families include A13–A16, A46, and A49–A70. These properties distinguish compromise before acceptance from compromise after generation, current epoch evidence from stale evidence, and a live key from an erased earlier epoch key.

All selected properties survived. In the modeled systems, compromise is therefore an explicit event with a temporal scope; it is not treated as a mechanism that silently changes an already established generation history.

7.6.4 Rollback, revocation, and transparency conditions remained explicit

The rollback, revocation, and transparency properties tested verifier-state regression, unauthorized certificate reactivation, acceptance with missing revocation state, conflicting checkpoint roots, split views, forked generalized acceptance, and conflicting transparency histories.

All selected attack classes survived. Where acceptance after an adverse event is allowed, the corresponding property requires the modeled exception or prior compromise rather than leaving the event unexplained.

7.6.5 Provenance injectivity held across seven model layers

A74–A80 tested the reverse direction of identity binding: whether the same complete provenance could justify two distinct Child identities.

All seven survived across the foundational, external-signature, claim-collapse, external, composition, signed-certificate, and C2/C3 composition theories.

Together with the direct lineage-binding properties, this supports both directions needed for a stable identity relation in the tested symbolic setting:

```
same accepted Child identity→
 same bound lineage, unless modeled compromise applies

same complete provenance→
 not two distinct Child identities
```

7.7 Computational Profile of the Selected Final Evidence

The per-target `processing time` values recorded by Tamarin in the selected final logs have the following profile:

```
Aggregate of selected per-target processing times: 2985.35 seconds
Median across all 88 targets: 2.64 seconds
Third quartile across all 88 targets: 8.94 seconds
Maximum selected target: 726.84 seconds
Median across attack targets: 2.42 seconds
Median across health targets: 4.06 seconds
```

The aggregate is a sum of per-target log values. It is not the initial parallel battery wall-clock duration and should not be interpreted as a benchmark for another machine or future verifier version.

The ten slowest selected final-evidence targets were:

A47 — 726.84 seconds
Kind: attack
Model: `oclm_phase1c3_forward_secure_epochs.spthy`
Evidence method: clean direct rerun

A48 — 707.76 seconds
Kind: attack
Model: `oclm_phase1c3_forward_secure_epochs.spthy`
Evidence method: integrated all-traces proof with three verified helper lemmas

A46 — 373.17 seconds
Kind: attack

Model: `oclm_phase1c3_forward_secure_epochs.spthy`
Evidence method: clean direct rerun

A14 — 301.59 seconds
Kind: attack
Model: `oclm_phase1c3_forward_secure_epochs.spthy`
Evidence method: clean direct rerun

A13 — 290.39 seconds
Kind: attack
Model: `oclm_phase1c3_forward_secure_epochs.spthy`
Evidence method: clean direct rerun

A04 — 69.98 seconds
Kind: attack
Model: `oclm_phase1c3b_c2_c3_composition_contract.spthy`
Evidence method: clean direct rerun

A43 — 59.75 seconds
Kind: attack
Model: `oclm_phase1b_external.spthy`
Evidence method: clean direct rerun

A80 — 34.13 seconds
Kind: attack
Model: `oclm_phase1c3b_c2_c3_composition_contract.spthy`
Evidence method: clean direct rerun

A08 — 34.08 seconds
Kind: attack
Model: `oclm_phase1c3b_c2_c3_composition_contract.spthy`
Evidence method: clean direct rerun

A16 — 33.58 seconds
Kind: attack
Model: `oclm_phase1c3b_c2_c3_composition_contract.spthy`
Evidence method: clean direct rerun

The five heaviest targets—A13, A14, A46, A47, and A48—are all associated with the C3 forward-secure epoch model. This concentration indicates that the most expensive part of the campaign was not basic identifier binding, but reasoning about historical acceptance, erased epochs, issuance ancestry, target-key compromise, and past-identity consistency.

7.8 The A48 Result in the Complete Set

A48 is one of the 80 attack classes, not an additional result beyond the total.

Its direct existential attack search did not complete within the practical resource budget. The final evidence therefore used a logically structured proof:

```
H1 same Child ID→
    same lineage

H2 same generated lineage→
    same public ancestry tuple

H3 uncompromised acceptance→
    matching generated origin

Final A48→
    past-epoch identity is bound
     or a modeled target key was compromised
```

All four properties were verified in the same Tamarin invocation, and all wellformedness checks succeeded. Section 8 analyzes that proof in detail.

7.9 Interpretation of Zero Counterexamples

"Zero counterexample traces" means that Tamarin did not find a trace satisfying any of the 79 direct prohibited `exists-trace` properties selected as final evidence, and the integrated A48 `all-traces` property was verified.

It does not mean:

- that no implementation defect can exist;
- that physical Oracle evidence is automatically truthful;
- that unmodeled cryptographic primitives cannot fail;
- that denial of service is impossible;
- that governance cannot misuse legitimate authority; or
- that all conceivable cross-chain systems inherit the result without a refinement argument.

The result is strongest when stated precisely:

> Under the stated Phase 1 and Phase 2 symbolic models, attacker capabilities, and compromise conditions, all 80 tested adversarial classes were ruled out, no counterexample trace was found, no attack class remained unresolved, and all eight representative health properties were reachable.

7.10 Result Artifacts

The publication package preserves the result analysis in both human-readable and machine-readable forms:

```
catalog/FINAL_88_RESULTS.csv
results/RESULTS_BY_ATTACK_FAMILY.csv
results/RESULTS_BY_MODEL.csv
results/RESULTS_PROFILE.json
results/SLOWEST_FINAL_EVIDENCE_TARGETS.csv
```

The authoritative per-property mapping remains `catalog/FINAL_88_RESULTS.csv`. The derived Section 7 tables can be regenerated from that file and do not replace the frozen evidence archive.

7.11 Summary

Phase 2 produced a complete result set across the 17 inherited Phase 1 models:

```
80 / 80 adversarial attack classes ruled out
0 counterexample traces
0 unresolved attack classes
8 / 8 health and reachability properties reachable
3 / 3 A48 helper lemmas verified in the integrated invocation
```

The result supports the tested form of canonical lineage non-substitutability: altering records, reusing identifiers, splicing ancestry evidence, rolling back verifier state, exploiting later compromise, or presenting conflicting transparency histories did not create a valid alternative canonical lineage under the modeled conditions.

8. Integrated Proof of A48: Past-Epoch Identity Binding

8.1 Purpose of A48

A48 is the final and computationally most difficult adversarial class in the Phase 2 campaign.

It asks whether two accepted certificates can refer to the same `child_id` under the same root while carrying different past-epoch identity components, without a relevant target epoch key or root key having been compromised before acceptance.

The property binds the accepted identity of a Child to the public tuple:

```
epoch
epoch public key
predecessor
```

```
issuer
lineage
root
Parent A
Parent B
timeA
timeB
timeC
validity window
space
```

The final property permits a mismatch only when one of the explicitly modeled compromise disjuncts applies.

Accordingly, A48 does not claim that forged evidence remains distinguishable after every possible key compromise. It states that, absent the relevant modeled compromise before either acceptance, the same accepted Child identity cannot carry two different bound public histories.

8.2 Why the Direct Attack Search Was Not the Final Evidence

The original adversarial formulation negated the desired universal property and searched for an existential counterexample trace.

The frozen original log completed theory loading, translation, derivation checks, and source saturation, but the attack search did not resolve within the configured 1,800-second limit:

```
AUDIT TIMEOUT AFTER 1800 SECONDS
```

A timeout is not a counterexample and is not a proof.

The intermediate Phase 2 state therefore classified A48 as unresolved rather than treating resource exhaustion as security evidence.

The final evidence changed the proof strategy, not the inherited C3 protocol rules. The security statement was decomposed into three reusable helper lemmas and a final universal property.

8.3 Integrated Theory

The final derived theory is:

```
models/a48/oclm_phase1c3_a48_decomposed_h1_h2_h3.spthy
```

SHA-256:

```
6a674d76a6c37479a10c727387a9a5771b7557064781ea4411b658e9bf355133
```

The theory inherits the C3 forward-secure epoch protocol and adds H1, H2, and H3 as proof obligations. It does not weaken the inherited acceptance, generation, signature, epoch, erasure, or compromise rules.

The integrated proof log is:

```
logs/clean/A48_integrated_H1_H2_H3_final.txt
```

SHA-256:

```
7a3b2b0e6b64e2d8bfca323dce4ca49c267fb78286044a11e4c39645a4a714c1
```

8.4 H1 — Same Child ID Implies the Same Lineage

H1 is:

```
a48_h1_same_child_id_implies_same_lineage
```

Its premise contains two `ForwardSecureChildAccepted` events under the same `root_pk` and the same `child_id`.

Its conclusion is:

```
lineage1 = lineage2
```

The purpose of H1 is to close the identity-to-lineage edge.

In the C3 model, the accepted Child identifier is checked against a lineage-bound construction. Under the symbolic term model, two accepted instances carrying the same bound Child identifier cannot detach that identifier from two different lineage terms.

Integrated result:

```
a48_h1_same_child_id_implies_same_lineage
(all-traces): verified (6 steps)
```

H1 alone does not yet prove equality of every public ancestry field. It establishes equality of the lineage terms to which the shared Child identity is bound.

8.5 H2 — Same Generated Lineage Implies the Same Public Tuple

H2 is:

```
a48_h2_same_generated_lineage_implies_same_public_tuple
```

Its premise contains two `ChildGeneratedInsideEpoch` events under the same `root_pk` and the same `lineage` .

Its conclusion equates:

```
epoch
epoch public key
predecessor
issuer
child_id
root
Parent A
Parent B
timeA
timeB
timeC
validity window
space
```

between the two generation events.

H2 therefore closes the lineage-to-ancestry edge:

```
same honestly generated lineage→
 same bound public ancestry tuple
```

Integrated result:

```
a48_h2_same_generated_lineage_implies_same_public_tuple
(all-traces): verified (2 steps)
```

The lemma is deliberately stated over generation events rather than acceptance events. This prevents accepted evidence from being treated as self-authenticating; the proof still requires a bridge from acceptance back to an actual matching generation event.

8.6 H3 — Uncompromised Acceptance Has a Matching Generated Origin

H3 is:

```
a48_h3_uncompromised_acceptance_has_generated_origin
```

Its premise contains one `ForwardSecureChildAccepted` event and excludes:

```
a matching epoch-key compromise before acceptance
a root-key compromise before acceptance
```

Its conclusion requires an earlier matching `ChildGeneratedInsideEpoch` event with the same:

```
root_pk
epoch
epoch public key
predecessor
issuer
child_id
lineage
root
Parents
times
window
space
```

H3 closes the acceptance-to-origin edge:

```
uncompromised accepted certificate→
 matching prior generation event
```

Integrated result:

```
a48_h3_uncompromised_acceptance_has_generated_origin
(all-traces): verified (52 steps)
```

H3 was the most expensive helper obligation because it traverses certificate acceptance, credential issuance, signature validity, epoch state, generation ancestry, and the temporal placement of compromise.

8.7 Final A48 Property

The final lemma is:

```
past_epoch_identity_is_bound_or_target_compromised
```

Its premise contains two accepted certificates with:

```
the same root_pk
the same child_id
```

The conclusion is a disjunction.

Identity-binding branch

Every bound public field is equal:

```
epoch1 = epoch2
epoch_pk1 = epoch_pk2
predecessor1 = predecessor2
issuer1 = issuer2
lineage1 = lineage2
root1 = root2
A1 = A2
B1 = B2
timeA1 = timeA2
timeB1 = timeB2
timeC1 = timeC2
window1 = window2
space1 = space2
```

Explicit compromise branches

Alternatively, one of the following occurred before either acceptance:

```
the first target epoch key was compromised
the second target epoch key was compromised
the root key was compromised
```

The compromise disjuncts are part of the security statement. They identify conditions under which the modeled evidence boundary no longer supports the uncompromised uniqueness conclusion.

Integrated result:

```
past_epoch_identity_is_bound_or_target_compromised
(all-traces): verified (8 steps)
```

8.8 Logical Composition

The proof can be read as a case analysis.

Assume two accepted certificates share the same `root_pk` and `child_id`.

Case 1 — A relevant compromise occurred

If either target epoch key or the root key was compromised before either acceptance, one of the explicit compromise disjuncts in the final property is satisfied.

No equality claim beyond the modeled evidence boundary is required in this branch.

Case 2 — No relevant compromise occurred

Assume none of the final compromise disjuncts applies.

This assumption is strong enough to satisfy the no-prior-compromise premises of H3 separately for each accepted certificate.

Applying H3 twice yields two matching earlier generation events.

The reasoning then proceeds as follows:

```
two accepted events share the same child_id↓
       H1
their accepted lineage terms are equal↓
       H3 applied to each acceptance
each acceptance has a matching prior generation event↓
       H2
the generated public ancestry tuples are equal↓

the identity-binding branch of final A48 holds
```

Therefore, whether the proof enters the compromise branch or the uncompromised branch, the final disjunction holds.

Schematically:

```
₁
Accepted(root_pk, child_id, ₁tuple)∧₂
 Accepted(root_pk, child_id, ₂tuple)↓

RelevantCompromisev

(
  H3(₁Accepted) ∧H3(₂Accepted)∧
   H1(child_id)∧
   H2(lineage)
)↓

RelevantCompromisev₁
 tuple = ₂tuple
```

This is the core formal connection between Child identity, generated lineage, accepted evidence, and past-epoch ancestry.

8.9 Same-Invocation Verification

The final Tamarin execution selected all four obligations:

```
H1
H2
H3
final A48
```

The integrated log reports:

```
a48_h1_same_child_id_implies_same_lineage
(all-traces): verified (6 steps)

a48_h2_same_generated_lineage_implies_same_public_tuple
(all-traces): verified (2 steps)

a48_h3_uncompromised_acceptance_has_generated_origin
(all-traces): verified (52 steps)

past_epoch_identity_is_bound_or_target_compromised
(all-traces): verified (8 steps)
```

It also reports:

```
All wellformedness checks were successful.
processing time: 707.76s
```

Verifying all four selected obligations in the same invocation closes the publication-artifact concern that the final A48 result might be accompanied only by unresolved helper obligations.

The recorded Tamarin exit status is:

```
NOT_CAPTURED_POSTPROCESS
```

This is not presented as zero because the Tamarin process completed and wrote the full verified log, but a later shell `awk` post-processing error occurred before the stored exit status was printed. The publication preserves that fact rather than reconstructing an unrecorded value.

8.10 What A48 Establishes

Within the C3 symbolic model and its compromise assumptions, A48 establishes:

> Two accepted certificates carrying the same Child identity under the same root cannot encode different past-epoch lineages and different bound public ancestry tuples unless a specified target epoch key or the root key was compromised before one of the acceptances.

This is stronger than saying that two serialized certificates are byte-identical.

It binds accepted Child identity to:

- lineage;
- epoch;
- epoch public key;
- predecessor;
- issuer;
- root;
- parents;
- temporal fields;
- validity window;
- and space.

It also distinguishes later compromise from an unexplained retroactive rewrite. Compromise must occur in the temporal position allowed by the final disjunction.

8.11 What A48 Does Not Establish

A48 does not establish that:

- physical Oracle observations are necessarily true;
- endpoint software cannot misrepresent an event before it enters the model;
- a compromised root key remains trustworthy;
- hash functions or signatures resist attacks outside the symbolic abstraction;
- every cross-chain bridge refines the C3 theory;
- all implementations preserve the exact field encodings used in the model;
- or denial of service is impossible.

A48 also does not prove end-to-end refinement from every Phase 1 lower-level theory into C6B.

Its contribution is precise: it closes the tested past-epoch substitution property in the C3 forward-secure epoch model.

8.12 Significance for Canonical Lineage Non-Substitutability

A48 is the clearest Phase 2 expression of the distinction between changing a record and changing the past of an entity.

An attacker may create another certificate, another ledger entry, another identifier claim, or another accepted history after a modeled compromise.

Without the relevant compromise, however, the tested model does not admit two different bound pasts under the same accepted Child identity.

The result can be summarized as:

```
same accepted Child identity
+ uncompromised evidence boundary→
 same generated lineage→
 same bound past-epoch ancestry tuple
```

This is the formal core of canonical lineage non-substitutability in the C3 model.

8.13 Reproducibility References

The frozen evidence archive contains:

```
models/a48/oclm_phase1c3_a48_decomposed_h1.spthy
models/a48/oclm_phase1c3_a48_decomposed_h1_h2.spthy
models/a48/oclm_phase1c3_a48_decomposed_h1_h2_h3.spthy
logs/clean/A48_integrated_H1_H2_H3_final.txt
source/A48_INTEGRATED_RUN_METADATA.txt
A48_HELPER_RESULTS.csv
FINAL_88_RESULTS.csv
```

The publication package additionally provides:

```
catalog/A48_HELPER_RESULTS.csv
results/A48_INTEGRATED_PROOF_SUMMARY.md
results/A48_PROOF_CHAIN.json
```

The archive hash and internal evidence hashes are recorded in `EVIDENCE_REFERENCE.md` and protected by the package-level `MANIFEST.sha256`.

8.14 Summary

The original direct A48 attack search timed out and was correctly left unresolved.

The final result did not reinterpret that timeout as success. Instead, it replaced one monolithic search obligation with three explicit helper lemmas and one final universal property.

The integrated run verified:

```
H1: same Child ID implies the same lineage
H2: the same generated lineage implies the same public tuple
H3: uncompromised acceptance has a matching generated origin
A48: past-epoch identity is bound or a target key was compromised
```

All wellformedness checks succeeded.

A48 therefore moved from an unresolved attack search to a structured, warning-free, integrated proof of the tested past-epoch identity-binding property.

9. OCLM as a Higher Layer Binding Multiple Blockchains

9.1 Position of OCLM

OCLM is not proposed as a replacement for blockchains.

A blockchain can provide:

- transaction ordering;
- replicated state;
- consensus over a chain-local history;
- signature and authorization checks;
- finality or probabilistic finality;
- economic settlement;
- and availability of recorded evidence.

These functions remain useful. Representative permissionless and BFT ledger designs illustrate these chain-local ordering and consensus roles [3,4].

OCLM addresses a different question:

> When several blockchains, ledgers, databases, or evidence systems refer to an entity, do their records belong to the same authenticated lineage?

Accordingly, OCLM is positioned above chain-local consensus.

```
Chain A consensus ┐
Chain B consensus ├──OCLM lineage layer
Chain C consensus ┘
```

Each chain may determine which records it accepts internally. OCLM evaluates whether accepted records across those systems preserve a common origin and authorized continuity.

9.2 Consensus and Canonical Lineage Are Different Relations

Blockchain consensus determines a chain-local state according to the rules of that blockchain.

Canonical lineage is a relation among:

- origin;
- parent or predecessor states;
- generation events;
- time;
- space or domain;
- context;
- authorization;
- and continuity.

These relations may coincide, but they are not identical.

A chain may reach consensus on a record that:

- was generated from the wrong predecessor;
- reuses an existing identifier;
- was minted without a valid migration event;
- represents a replayed bridge message;
- follows a compromised authority;
- or belongs to a different lineage.

The chain may still be internally consistent.

OCLM therefore treats consensus as one possible source of evidence, not as a mechanism that can redefine origin by itself.

```
consensus-selected record≠
 automatically canonical successor
```

9.3 The Higher-Layer Binding Model

A multi-chain OCLM deployment can be viewed as four logical layers.

Layer 1 — Chain-local execution

Each blockchain processes transactions and maintains its own local state.

Examples include:

- ownership records;
- token balances;
- burn or lock events;
- credential issuance;
- revocation state;
- application state;
- and governance decisions.

Layer 2 — Evidence extraction and attestation

Relevant chain-local events are converted into authenticated evidence.

Evidence may include:

- finalized transaction inclusion;
- block or checkpoint references;
- state proofs;
- authenticated event logs;
- threshold attestations;
- hardware attestations;
- or signed Oracle statements.

Layer 3 — OCLM lineage validation

OCLM evaluates whether the evidence establishes:

- the correct origin;
- the correct parent or predecessor;
- the required temporal order;
- the correct source and destination domains;
- the required authorization context;
- and uninterrupted canonical continuity.

Layer 4 — Application and settlement action

A destination application decides what to do with the validated lineage result.

Possible actions include:

- minting or activating a successor representation;
- accepting a credential;
- transferring control;
- releasing escrow;
- recognizing ownership;
- or rejecting a substituted lineage.

This separation allows blockchains to retain their native consensus and execution models while OCLM supplies a common lineage criterion above them.

9.4 Canonical Cross-Chain Migration

Consider a Child represented on Blockchain A that is to continue on Blockchain B.

A canonical migration requires more than the appearance of a similar identifier on Blockchain B.

At minimum, the lineage relation must connect:

1. the source Child and its current lineage on Blockchain A;
2. an authorized migration, lock, burn, transfer, or succession event;
3. the relevant source-chain evidence;
4. the destination generation event;
5. the resulting destination Child or successor state;
6. and the continuity binding the source and destination states.

Conceptually:

```
Child_A||

  authorized migration evidence▼

Transition||

  destination generation▼

Child_B
```

The destination representation may differ in serialization, contract address, token format, or storage system.

It remains canonical only if the transition preserves authenticated continuity.

Thus:

```
different representation
+ authenticated continuity
= canonical migration
```

whereas:

```
same visible identifier
+ no authenticated continuity
= substitution claim
```

9.5 Migration Does Not Require a Single Global Chain

OCLM does not require all participating blockchains to merge into one consensus network.

Blockchain A may use one consensus mechanism.
Blockchain B may use another.
Blockchain C may be permissioned.
A separate evidence system may not be a blockchain at all.

The common element is not a shared block-production mechanism.

The common element is the lineage relation.

```
heterogeneous ledgers
+ authenticated evidence
+ OCLM continuity rules
= higher-order lineage
```

This permits a canonical lineage to pass through multiple technological domains without claiming that those domains have become one ledger.

9.6 Chain Reorganizations and Finality

A chain-local reorganization changes which records a blockchain currently recognizes.

OCLM must therefore distinguish between:

- provisional evidence;
- finalized or sufficiently stable evidence;
- later-invalidated evidence;
- and evidence already used to generate a successor state.

A concrete deployment must define the point at which a chain-local event becomes eligible as a lineage parent.

Possible policies include:

- deterministic finality;
- a confirmation-depth rule;
- a checkpoint accepted by a trusted or threshold authority;
- or a finality proof from the source protocol.

OCLM does not create finality where the underlying chain provides none. It makes the finality assumption explicit within the lineage evidence.

A reorganization that removes the asserted source event may therefore invalidate a pending migration or create a dispute requiring a defined recovery rule.

9.7 Forks and Competing Chain Histories

When a blockchain forks, more than one chain-local history may claim continuity from the same earlier state.

OCLM does not resolve every fork merely by choosing the longest or economically strongest branch.

Instead, a deployment must define which fork evidence satisfies the lineage policy.

Relevant criteria may include:

- protocol-native finality;
- checkpoint authority;
- governance rules;
- revocation state;
- non-equivocation evidence;
- and continuity with already accepted successor states.

The OCLM contribution is to prevent a branch from becoming the canonical history of a Child solely because it repeats the Child identifier.

A competing branch must establish the required lineage continuity under the applicable policy.

9.8 Bridge Messages Are Evidence, Not Identity

A bridge message may state that an asset was locked, burned, transferred, or released. Existing cross-chain research includes atomic swaps, cryptocurrency-backed asset protocols, and systematic analyses of communication across distributed ledgers [7–9].

The message is evidence concerning a transition.

It is not itself the canonical identity of the asset.

A secure cross-chain design must bind the message to:

- the source Child;
- source lineage;
- source-chain event;
- source and destination domains;
- transition nonce or single-use resource;
- authorized destination;
- destination generation;
- and resulting successor lineage.

Without these bindings, an otherwise authentic bridge message may be:

- replayed;
- redirected;
- reused in another domain;
- combined with evidence from another instance;

- or applied to the wrong Child.

The Phase 2 replay, cross-instance, time-space, and composition attack families directly motivate these requirements.

9.9 Multiple Representations of One Lineage

A canonical lineage may have more than one representation when the application explicitly permits it.

Examples include:

- a primary asset and a read-only mirror;
- a legal record and a technical representation;
- a credential displayed in several wallets;
- or synchronized states across several systems.

OCLM does not require every representation to be the same object.

It requires the relationship among representations to be explicit.

A deployment must distinguish:

- canonical successor;
- authorized mirror;
- derivative representation;
- temporary escrow representation;
- and unrelated substitute.

These roles must not be inferred solely from identifier equality.

9.10 One Canonical Successor and Authorized Multiplicity

Some applications require exactly one active successor.

Others permit several authorized descendants or views.

OCLM can support either policy, but the multiplicity rule must be part of the generation context.

For a single-successor asset:

```
one valid transition→
 one active canonical successor
```

For an authorized multi-child composition:

```
one parent lineage
+ explicit multi-child policy→
 several authorized descendants
```

The security objective is not to impose universal uniqueness.

It is to prevent an attacker from converting unauthorized multiplicity into canonical continuity.

9.11 Revocation Across Chains

Revocation is difficult when evidence has already propagated to several systems.

An OCLM-based design should bind revocation to:

- the authority entitled to revoke;
- the affected lineage state;
- the effective time or event order;
- the applicable domains;
- and the consequences for descendants.

A destination chain that accepts a stale pre-revocation proof may remain internally consistent while violating the intended cross-chain lineage policy.

The Phase 1 revocation and verifier-monotonicity models, together with the Phase 2 stale-state and unauthorized-reactivation attack classes, establish the formal basis for rejecting such acceptance under the tested rules.

A concrete deployment must still define propagation, latency, recovery, and dispute handling.

9.12 Transparency and Split-View Resistance

A cross-chain lineage service may publish checkpoints, roots, or evidence summaries.

If different verifiers receive conflicting views, an attacker may attempt to establish incompatible canonical histories.

OCLM therefore benefits from transparency mechanisms related to append-only logging, consistency proofs, and detection of conflicting views, as exemplified by Certificate Transparency [5]. Such mechanisms may provide:

- globally comparable checkpoints;
- consistency proofs;
- non-equivocation evidence;
- verifier memory;
- and monotonic acceptance rules.

The Phase 1 transparency and split-view theories model these concerns symbolically.

They do not mandate one specific transparency-log implementation, but they demonstrate that canonical lineage should not depend on an unverifiable private view supplied independently to each verifier.

9.13 Root and Epoch-Key Structure

A practical multi-chain deployment may use a long-lived root authority together with shorter-lived epoch keys.

The root can authorize epoch succession.

Epoch keys can issue or attest lineage evidence for bounded periods or domains.

This structure supports:

- key rotation;
- reduced exposure;
- forward-security reasoning;
- compartmentalization by chain or application;
- and explicit compromise windows.

A later epoch-key compromise need not automatically rewrite evidence generated under an earlier erased epoch key.

This temporal separation is one of the central properties tested by the C3 model and the A48 proof.

9.14 OCLM and Existing Blockchain Assets

OCLM can be applied without discarding existing blockchain assets.

A deployment may attach OCLM lineage evidence to:

- fungible tokens;
- non-fungible tokens;
- credentials;
- real-world asset records;
- supply-chain objects;
- legal documents;
- identity claims;

- governance rights;
- or application-specific states.

The existing blockchain remains responsible for chain-local execution and settlement.

OCLM supplies an additional answer:

> Is this record a canonical continuation of the lineage it claims?

This allows existing blockchains to become components of a broader lineage system rather than isolated sources of absolute identity.

9.15 Minimal Cross-Chain Lineage Record

A concrete cross-chain lineage record may include fields such as:

```
lineage version
source domain
source chain identifier
source object identifier
source state or transaction reference
source finality evidence
source lineage identifier
transition type
transition nonce
authorized destination domain
destination generation evidence
destination object identifier
predecessor lineage identifier
time or event-order constraints
revocation and transparency references
issuer or attesting authority
signature or threshold proof
```

This is an architectural template, not a frozen OCLM wire format.

A future implementation profile must specify:

- canonical serialization;
- domain separation;
- cryptographic algorithms;
- proof formats;
- finality rules;
- verifier state;
- revocation distribution;
- and failure recovery.

9.16 Verification Boundary

The Phase 2 result does not yet prove a complete deployed cross-chain system.

The current formal evidence establishes properties of the Phase 1 Tamarin theories and the Phase 2 adversarial battery.

A concrete end-to-end claim would additionally require:

1. a precise implementation architecture;
2. a mapping from implementation events to formal events;
3. proof that the implementation preserves the modeled rules;
4. proof that source-chain and destination-chain evidence satisfy the assumed predicates;
5. and refinement or composition arguments connecting lower-level models to the generalized contract.

In particular, the C6B generalized security-composition contract is an abstract assume-guarantee layer. It is not yet a machine-checked refinement proof connecting every Phase 1 model and every concrete blockchain implementation end to end.

This boundary does not weaken the Phase 2 result. It identifies the next formalization layer required for deployment-specific assurance.

9.17 Architectural Invariants

A concrete OCLM cross-chain profile should preserve at least the following invariants.

Origin binding

Every successor must remain traceable to the authenticated origin required by the application.

Predecessor binding

A successor must identify the correct prior lineage state.

Domain binding

Evidence from one chain, network, jurisdiction, or application domain must not be silently reusable in another.

Transition uniqueness

A single-use migration event must not authorize multiple incompatible successors unless the policy explicitly permits multiplicity.

Temporal ordering

Generation, compromise, revocation, migration, and acceptance must retain their required event order.

Context binding

Evidence from unrelated executions must not be composable into a valid new lineage.

Monotonic verification

A verifier must not accept stale state after learning a later valid state where the policy requires monotonicity.

Explicit compromise

Acceptance outside the normal lineage relation must be explainable only through a modeled compromise or governance exception, not through an unexplained substitution.

9.18 Example: Three-Chain Lineage

Consider one asset that moves from Blockchain A to Blockchain B and later to Blockchain C.

```
Origin on A||

   Transition AB▼

Successor on B||

   Transition BC▼

Successor on C
```

A chain-local view may contain three different contract addresses and three different object identifiers.

The OCLM view contains one lineage with two authorized transitions.

For the representation on C to be canonical, the evidence must connect:

```
Origin_A→
 Transition_AB→
 Generation_B→
 Transition_BC→
 Generation_C
```

Creating a new object on C with the same visible label but without this chain of authenticated continuity produces another object, not the canonical successor.

9.19 Security Interpretation

The higher-layer interpretation of OCLM can be summarized as follows.

Blockchains answer:

```
What state did this chain accept?
```

OCLM asks:

```
What authenticated lineage does this state belong to?
```

A blockchain can protect the integrity of its own record sequence.

OCLM is intended to protect the identity relation that crosses those sequences.

Therefore:

```
blockchain integrity
+ OCLM lineage continuity
= stronger cross-system identity assurance
```

The two mechanisms are complementary.

9.20 Summary

OCLM does not require the abandonment of existing blockchains.

It allows multiple blockchains and evidence systems to remain independent while participating in one authenticated lineage.

The essential distinction is:

```
chain-local consensus≠
 cross-system canonical lineage
```

A canonical migration may change chain, representation, contract, or storage location.

It must not change origin and continuity without creating a different lineage.

This is the architectural role of OCLM as a higher layer binding multiple blockchains.

10. Assumptions, Limitations, and Formal Boundary

10.1 Purpose of This Section

The Phase 2 result is strongest when its boundary is explicit.

Formal verification does not become weaker by stating its assumptions. It becomes meaningful because readers can determine exactly which claims were established, which conditions were modeled, and which obligations remain for implementation and deployment.

This section therefore defines:

- the symbolic cryptographic assumptions;
- the event and compromise assumptions;
- the relationship between records and external truth;
- the scope of the 80 attack classes;
- the limits of health and reachability checks;

- the status of the C6B composition contract;
- and the additional work required for end-to-end deployment assurance.

10.2 Symbolic Cryptographic Model

The Phase 1 and Phase 2 analyses use the symbolic model supported by the Tamarin Prover.

Cryptographic operations are represented by terms and equations rather than by probabilistic algorithms operating over concrete bit strings.

Within the applicable theories, the model assumes that:

- signatures cannot be forged without the relevant signing capability;
- hashes behave according to the declared symbolic construction;
- private values remain unavailable unless disclosed through a rule;
- domain-separated constructors remain distinct;
- and the adversary can combine, replay, decompose, and transmit terms only according to the declared algebra.

This abstraction is appropriate for protocol-logic analysis.

It does not itself prove:

- concrete computational security bounds;
- resistance to implementation-specific side channels;
- resistance to fault injection;
- resistance to memory disclosure;
- security of random-number generation;
- or security against future cryptanalytic advances.

A deployment must select concrete primitives whose computational properties justify the symbolic assumptions.

10.3 Free-Constructor and Hash-Binding Assumptions

Several lineage properties rely on structured symbolic terms and domain-separated hash constructions.

For example, the C3 model binds lineage and Child identity through symbolic hash terms.

Under the symbolic model:

```
equal constructed hashes→
 equal underlying constructor terms
```

where no collision rule is provided.

This is the basis for properties such as:

```
same Child ID→
 same bound lineage
```

The result should therefore be interpreted as a symbolic binding property.

A concrete implementation must additionally ensure:

- collision-resistant hash functions;
- canonical serialization;
- unambiguous field boundaries;
- domain separation;
- stable encoding across implementations;
- and rejection of malformed or non-canonical encodings.

Without those implementation properties, a concrete system might violate the assumptions represented by the symbolic constructor.

10.4 Signature and Key Assumptions

The models treat signatures as evidence that the holder of the relevant signing capability authenticated the encoded statement.

A valid signature does not automatically establish that:

- the signer was honest;
- the signing device was uncompromised;
- the external facts were physically true;
- the key was used under the correct policy;
- or the signer understood the statement.

These concerns are represented through explicit compromise events, authorization rules, generation events, and model-specific acceptance predicates.

The security results therefore depend on the completeness of the modeled key roles and compromise rules.

A concrete deployment must identify all authorities whose compromise can affect lineage acceptance, including:

- roots;
- epoch issuers;
- revocation authorities;
- transparency authorities;
- Oracle operators;
- bridge or migration authorities;
- hardware roots;
- and governance-controlled emergency keys.

10.5 Temporal Compromise Boundary

Many Phase 2 properties distinguish compromise before acceptance from compromise after generation or after acceptance.

This distinction is central.

The model does not treat key compromise as a metaphysical rewrite of prior events.

Instead, compromise affects which later messages the adversary can create and which acceptance conclusions remain justified.

The exact security statement depends on the temporal relation encoded in each lemma.

For example:

```
compromise before acceptance
```

may satisfy an explicit exception branch, while:

```
compromise after an earlier protected generation
```

does not automatically establish that the earlier event had a different lineage.

A deployment must preserve trustworthy event ordering.

If timestamps, checkpoints, revocation state, or epoch transitions can be reordered by implementation behavior, the concrete system may fail to satisfy the temporal assumptions used in the proof.

10.6 Key Erasure Assumption

Forward-security reasoning depends on the modeled erasure of prior epoch secrets.

The symbolic model can represent that a previous key is no longer available after an epoch transition.

The proof does not verify physical erasure from:

- memory;
- backups;
- logs;
- crash dumps;
- secure enclaves;
- hardware devices;
- operator archives;
- or disaster-recovery systems.

A concrete forward-secure deployment must implement and audit key lifecycle controls that justify the modeled erasure event.

If an old epoch secret remains recoverable, the implementation does not refine the corresponding formal assumption.

10.7 Oracle and External-Truth Boundary

OCLM binds authenticated evidence concerning origin, parents, time, space, context, and continuity.

It does not create external truth from nothing.

An Oracle may attest that an event occurred at a certain time or place. The formal model can verify how that attestation is bound into lineage and whether the adversary can substitute another value under the stated rules.

The model does not independently verify that the physical event actually occurred.

Therefore:

```
formally valid Oracle evidence≠
 automatic proof of physical truth
```

The external-truth boundary may depend on:

- sensor integrity;
- human observation;
- legal authority;
- hardware attestation;
- threshold governance;
- source-chain finality;
- or another external evidence process.

OCLM protects the authenticated relationship among the modeled claims. The trustworthiness of the initial external observation must be justified separately.

10.8 Records, Evidence, and Ontological Truth

The conceptual OCLM theory distinguishes records from the history they describe.

The formal model does not claim omniscient access to ontological truth.

It defines a canonical lineage relative to authenticated generation and continuity predicates represented in the theory.

Accordingly, the formal statement is:

> The tested model prevents substitution of a different lineage under the same accepted Child identity unless a modeled exception or compromise condition applies.

It is not:

> The verifier has perfect knowledge of every physical fact in the universe.

This distinction preserves the conceptual claim while keeping the machine-verified statement exact.

10.9 Scope of the 80 Attack Classes

The 80 attack classes were deliberately constructed to challenge the inherited Phase 1 properties across:

- lineage and identity substitution;
- cross-instance ancestry splicing;
- replay and stale evidence;
- rollback;
- key compromise timing;
- revocation;
- transparency;
- split views;
- generalized composition;
- and provenance injectivity.

They provide broad symbolic coverage of the modeled security objectives.

They are not a claim that exactly 80 attacks exist in the world.

They do not exhaust:

- implementation vulnerabilities;
- economic attacks;
- governance capture;
- denial of service;
- privacy leakage;
- traffic analysis;
- unsafe user interfaces;
- deployment misconfiguration;
- unmodeled cryptographic APIs;
- or attacks against external dependencies.

The correct claim is:

```
80 tested symbolic adversarial classes
80/80 ruled out under the stated models and assumptions
```

10.10 Meaning of No Counterexample Trace

For 79 direct attack properties, Tamarin found no trace satisfying the prohibited existential condition.

For A48, the equivalent universal security property was verified through the integrated H1/H2/H3 proof.

“No counterexample trace” means that no violating trace exists in the analyzed symbolic transition system for the selected property.

It does not mean that:

- every implementation is correct;
- every possible model extension preserves the property;
- every physical deployment is secure;
- or every future threat has already been represented.

Changing the rules, equations, compromise events, acceptance predicates, or external assumptions creates a different formal problem that must be analyzed separately.

10.11 Health and Reachability Boundary

The eight health properties demonstrate that representative intended executions are reachable.

This helps rule out a common form of vacuity in which a security property holds only because the protocol cannot execute.

The health properties do not establish:

- universal liveness;
- guaranteed termination;
- fairness;
- availability under denial of service;
- bounded latency;
- throughput;
- economic viability;
- or usability.

They show that selected honest and adverse lifecycle patterns remain executable in the formal theories.

10.12 Soundness Relative to the Tamarin Models

The publication evidence establishes results for the exact model files identified by their SHA-256 hashes.

If a model file changes, the previous result does not automatically apply.

Relevant changes include:

- modified rules;
- added equations;
- altered compromise events;
- weaker acceptance checks;
- changed field bindings;
- modified restrictions;
- or additional adversary outputs.

The package therefore preserves:

- model files;
- proof logs;
- result mappings;
- metadata;
- and manifests.

These artifacts define the exact analyzed object.

10.13 Implementation-Refinement Boundary

A formally verified protocol model is not identical to its software implementation.

A concrete implementation may introduce:

- parsing ambiguity;
- incorrect serialization;
- missing domain separation;
- race conditions;
- state rollback;
- incomplete revocation checks;
- unsafe key storage;
- inconsistent clock handling;
- identifier truncation;
- database uniqueness failures;
- or acceptance paths absent from the model.

To transfer the formal result to an implementation, a refinement argument is required.

At minimum, the implementation must show that:

1. every security-relevant implementation event maps to a formal event;
2. every accepted implementation state satisfies the modeled acceptance predicates;
3. concrete cryptographic encodings preserve symbolic term distinctions;
4. persistent state preserves modeled monotonicity and single-use rules;

5. compromise and recovery behavior match the declared assumptions;
6. and no additional acceptance path bypasses the verified protocol.

Without this mapping, the formal result remains evidence about the design model rather than a complete software-security certification.

10.14 Concurrency and Distributed-State Assumptions

Tamarin models concurrent protocol traces symbolically.

A concrete distributed implementation must still preserve the intended atomicity and ordering of:

- single-use transitions;
- revocation updates;
- epoch changes;
- predecessor consumption;
- transparency checkpoints;
- and migration acceptance.

Database isolation, replication lag, message queues, retries, and failover behavior may create states not represented by a simplified protocol rule.

A deployment must ensure that concurrent implementation behavior refines the atomic or ordered events assumed by the model.

10.15 Finality and External Ledger Assumptions

When a blockchain or ledger is used as an Oracle or evidence source, OCLM depends on a defined finality predicate.

The formal models do not prove the consensus security of every external blockchain.

A deployment must specify:

- which chain state is accepted;
- how finality is determined;
- how reorganizations are handled;
- what happens after catastrophic consensus failure;
- and whether governance can override finalized history.

The lineage result is conditional on the accepted external-ledger evidence satisfying these predicates.

10.16 Revocation Distribution Assumption

Formal revocation properties assume that the verifier receives and applies the relevant revocation state according to the model.

A real deployment must address:

- propagation delay;
- offline verifiers;
- stale caches;
- partitioned networks;
- recovery after missed updates;
- and conflicting revocation authorities.

A verifier that never receives the revocation information may not satisfy the same assumptions as the formal verifier.

10.17 Transparency-System Assumption

The transparency theories model checkpoint, consistency, fork, and split-view behavior at a symbolic level.

They do not select or verify one production transparency implementation.

A concrete system must define:

- log data structures;
- append-only proofs;
- checkpoint signatures;
- gossip or witness mechanisms;
- consistency verification;
- storage retention;
- and recovery from equivocation.

The Phase 2 result supports the modeled non-equivocation and monotonicity requirements. It does not certify an unspecified log service.

10.18 Governance and Authorized Exceptions

Some real systems permit emergency recovery, judicial intervention, administrative reassignment, or governance-approved migration.

Such actions may be legitimate, but they change the lineage policy.

OCLM should represent them as explicit transitions or exception authorities rather than as silent rewrites.

An authorized exception may create:

- a successor lineage;
- a replacement credential;
- a recovery branch;
- or an administrative override.

It must not be described as unchanged canonical continuity unless the policy formally defines it that way.

The distinction between explicit governance transition and hidden substitution is essential.

10.19 Privacy Is Not Established by Phase 2

Phase 2 focuses on integrity, lineage, continuity, compromise boundaries, revocation, and transparency.

It does not establish privacy properties such as:

- anonymity;
- unlinkability;
- confidentiality of lineage fields;
- selective disclosure;
- resistance to traffic analysis;
- or minimization of personal data.

Because lineage evidence may contain rich historical relationships, privacy must be designed independently.

Possible approaches include:

- commitments;
- zero-knowledge proofs;
- selective-disclosure credentials;
- encrypted evidence;
- access-controlled attestations;
- and privacy-preserving accumulators.

These mechanisms require their own threat models and proofs.

10.20 Availability and Denial of Service

The verified properties do not guarantee availability.

An adversary may still:

- block messages;
- delay evidence;
- prevent checkpoint propagation;
- exhaust computational resources;
- attack Oracle infrastructure;
- or make a destination service unavailable.

OCLM can distinguish valid from invalid lineage evidence under its model. It does not force a verifier to remain online or guarantee that valid evidence will be processed within a deadline.

10.21 Economic and Incentive Assumptions

The models do not analyze incentives, bribery, market manipulation, validator economics, bridge liquidity, insurance, or governance-token concentration.

A technically valid authority may still act against the interests of users.

Economic security must therefore be evaluated separately from symbolic protocol correctness.

10.22 C6B Assume-Guarantee Boundary

The Phase 1 C6B model represents a generalized security-composition contract.

It expresses an abstract assume-guarantee relationship among security components.

It is not yet a machine-checked end-to-end refinement proof showing that:

- each lower-level Phase 1 theory;
- every concrete Oracle;
- every concrete blockchain;
- and every production implementation

satisfies the C6B assumptions and jointly realizes its guarantees.

The C6B result is therefore a verified abstract composition statement under its own rules.

A future refinement phase should connect:

```
concrete component model→
 component guarantee→
 C6B assumption→
 composed OCLM guarantee
```

This is the clearest remaining formal boundary between the Phase 2 model suite and a complete deployment-specific proof.

10.23 Trusted Computing Base

The effective trusted computing base for the Phase 2 evidence includes:

- the correctness of the Tamarin Prover and its dependencies;
- the correctness of the Tamarin model files;
- the declared equational theories;
- the operating environment used to run the proofs;
- the integrity of the captured logs;
- the SHA-256 implementation used for artifact identification;
- and the human interpretation connecting conceptual claims to formal lemmas.

The frozen evidence and manifests reduce the risk of artifact substitution.

They do not provide a formal proof of the verifier implementation itself.

10.24 Reproducibility Boundary

The publication package supports two kinds of reproducibility.

Artifact-level reproducibility

A reviewer can verify:

- the frozen evidence ZIP;
- internal hashes;
- result counts;
- model-log mappings;
- and package manifests.

Proof-level reproducibility

A reviewer can rerun Tamarin against the preserved theories with a compatible environment.

Exact processing times may differ because of:

- hardware;
- operating system;
- tool version;
- scheduling;
- and proof-search behavior.

The security result should be compared by lemma outcome and evidence identity, not by requiring identical wall-clock time.

10.25 Claims Supported by Phase 2

Phase 2 supports the following publication claim:

> Under the stated OCLM Phase 1 and Phase 2 symbolic models, attacker capabilities, cryptographic abstractions, and compromise conditions, all 80 tested adversarial attack classes were ruled out, no counterexample trace was found, no attack class remained unresolved, and all eight representative health and reachability properties were reachable.

It also supports the more focused conceptual claim:

> In the tested models, changing, duplicating, replaying, or re-recording evidence does not create a different canonical lineage under the same accepted Child identity unless an explicit modeled compromise or exception condition applies.

10.26 Claims Not Supported by Phase 2

Phase 2 does not support statements such as:

- OCLM is impossible to break;
- every possible attack has been tested;
- every blockchain bridge using OCLM is secure;
- OCLM automatically discovers physical truth;
- the implementation is verified merely because the model is verified;
- the 80 attack classes represent a statistical security probability;
- or C6B already constitutes a complete end-to-end implementation proof.

These stronger statements would exceed the evidence.

10.27 Future Formal Obligations

The principal future obligations are:

End-to-end refinement

Connect concrete component models to the C6B assume-guarantee contract.

Implementation model

Define a reference architecture and prove that its state transitions refine the symbolic rules.

Concrete cryptographic profile

Specify canonical serialization, algorithms, key sizes, domain separation, and proof formats.

Oracle profiles

Define how physical, legal, hardware, and blockchain evidence satisfy OCLM generation predicates.

Cross-chain finality profiles

Formalize source-chain finality, reorganization handling, and destination activation.

Privacy model

Add confidentiality, selective disclosure, and unlinkability properties where required.

Recovery and governance model

Represent authorized recovery without collapsing the distinction between succession and substitution.

These obligations are extensions of the current work, not corrections to the completed Phase 2 result.

10.28 Formal Boundary Summary

The Phase 2 evidence establishes a verified symbolic result for exact preserved theories.

Its boundary can be summarized as:

```
Verified:
protocol-level lineage properties
under declared symbolic and compromise assumptions

Not yet verified:
every physical input
every implementation
every external blockchain
every governance process
and complete end-to-end refinement
```

This boundary preserves both sides of the result:

- the verification is substantive and complete for the stated 88 targets;
- and the publication does not claim more than the evidence establishes.

10.29 Conclusion

The limitations identified in this section do not reduce the completed result to a mere proposal.

Phase 2 contains an executable formal model suite, an adversarial validation battery, clean final evidence, integrated resolution of A48, and reproducible artifact hashes.

The limitations define where the proof ends.

They also identify the exact path from a verified theory of canonical lineage non-substitutability toward a formally connected multi-blockchain implementation.

11. Conclusion and Final Phase 2 Statement

11.1 Conclusion

The Oracle–Child Lineage Model begins from a distinction that ordinary record integrity does not fully express:

> A record describing an entity is not identical to the authenticated lineage through which that entity arose.

A ledger can preserve a record.
A consensus protocol can select a state.
A signature can authenticate a statement.
A database can retain an identifier.

None of these mechanisms, by itself, establishes that a substituted record possesses the same origin, ancestry, temporal-spatial conditions, generation context, and continuity as the entity it claims to represent.

OCLM addresses this gap by binding Child identity to canonical lineage.

Phase 1 constructed the formal foundation across 17 Tamarin models, including:

- origin and ancestry binding;
- external signatures;
- time and space;
- claim collapse;
- signed composition;
- forward-secure epochs;
- verifier monotonicity;
- rollback resistance;
- revocation;
- transparency;
- split-view resistance;
- multi-epoch generalization;
- and an abstract generalized composition contract.

Phase 2 did not merely restate those properties.

It attempted to break them.

The adversarial campaign defined 80 symbolic attack classes and eight health and reachability properties. The attack battery examined lineage substitution, ancestry splicing, replay, stale state, rollback, compromise timing, unauthorized reactivation, transparency forks, conflicting canonical histories, evidence reuse, and provenance collisions.

The final result is:

```
Symbolic adversarial attack classes: 80
Attack classes ruled out: 80 / 80
Counterexample traces found: 0
Unresolved attack classes: 0

Health and reachability properties: 8
Reachable properties: 8 / 8

A48 helper lemmas: 3
Verified helper lemmas: 3 / 3
```

The final A48 property was resolved through an integrated proof establishing that:

```
same accepted Child identity
+ no relevant prior target compromise→
 same lineage→
 same generated public ancestry tuple
```

The helper lemmas and the final property were verified in the same Tamarin invocation, and all wellformedness checks succeeded.

11.2 Central Finding

The central Phase 2 finding is:

> Under the stated symbolic models, attacker capabilities, and compromise conditions, changing, duplicating, replaying, re-recording, or recombining evidence did not create a different canonical lineage under the same accepted Child identity.

This result does not depend on one blockchain being globally authoritative.

It depends on the preservation of authenticated origin and continuity.

Accordingly, OCLM can be positioned as a higher lineage layer above multiple blockchains, ledgers, and evidence systems.

Each system may preserve its own local history.

OCLM asks whether those local histories belong to the same canonical lineage.

11.3 Records Can Change Without Changing Origin

A record can be altered.

A database can be rewritten.

A blockchain can fork.

A key can later be compromised.

A replacement certificate can be issued.

A majority can accept another state.

These events can change what a system currently records or recognizes.

They do not, by themselves, make a different generation history become the original generation history of an existing Child.

Within OCLM, the distinction is represented formally:

```
record mutation≠
 lineage identity
```

and:

```
accepted replacement≠
 canonical continuity
```

unless the applicable generation, succession, or compromise conditions establish that relation.

This is the theoretical and formal center of canonical lineage non-substitutability.

11.4 Migration and Substitution

Phase 2 also clarifies the difference between legitimate migration and substitution.

A legitimate migration may change:

- blockchain;
- contract;
- identifier representation;
- storage system;
- jurisdiction;
- protocol version;
- or operational authority.

It remains canonical when authenticated continuity is preserved.

A substitution may copy the visible representation while lacking that continuity.

Therefore:

```
different representation
+ authenticated continuity
= canonical migration
```

whereas:

```
similar or identical representation
+ different lineage
= substitution
```

This distinction allows OCLM to support movement across systems without reducing identity to permanent residence on one ledger.

11.5 Relationship to Blockchain

OCLM does not reject blockchain.

It gives blockchain a precise position within a larger structure.

A blockchain can answer:

```
What state did this chain accept?
```

OCLM asks:

```
What authenticated lineage does this state belong to?
```

The combination is stronger than either relation alone:

```
chain-local integrity
+ cross-system lineage continuity
= higher-order identity assurance
```

OCLM therefore treats blockchains as powerful evidence and settlement systems, while refusing to equate consensus alone with canonical origin.

11.6 Meaning of the Formal Result

The formal result is substantive.

It consists of:

- executable Tamarin theories;
- explicitly stated adversarial properties;
- health and reachability checks;
- warning-free selected evidence;
- model and log hashes;
- a complete 88-target result map;
- and an integrated proof of the final difficult property.

At the same time, its scope remains exact.

The result applies to the preserved symbolic models.

It does not automatically prove:

- every physical Oracle;
- every software implementation;
- every cryptographic library;
- every blockchain;
- every governance system;
- or every possible attack not represented by the formal theories.

This boundary is not a retreat from the result.

It is the condition that makes the result scientifically and technically usable.

11.7 Completed Phase 2 Contribution

Phase 2 contributes five completed outcomes.

1. Adversarial validation

The Phase 1 lineage properties were challenged by 80 explicit symbolic attack classes.

2. Non-vacuity evidence

Eight representative health and reachability properties were shown to be reachable.

3. Complete final resolution

The final evidence contains:

```
80 / 80 attack classes ruled out
0 counterexamples
0 unresolved
```

4. Integrated A48 closure

The last unresolved search was replaced by a structured proof whose helper obligations and final property were verified together.

5. Multi-blockchain interpretation

The formal foundations were connected to an architecture in which independent chains can be bound by a higher canonical-lineage relation without being replaced by one universal chain.

11.8 Reproducibility and Evidence Preservation

The completed Phase 2 evidence is preserved through:

- exact model files;
- exact proof logs;
- result CSVs;
- validation metadata;
- SHA-256 manifests;
- and a frozen evidence archive.

The authoritative result is not a prose claim detached from its source.

It is connected to machine-readable evidence.

The frozen archive is:

```
oclm-phase2-final-evidence-20260721T050537Z.zip
```

SHA-256:

```
de36e384b7583ddde8dfc655ac51eeb517bb64011b5cf4d59a850aad62abb51e
```

The publication package preserves this archive without modifying it.

11.9 Final Phase 2 Statement

The official Phase 2 result statement is:

> OCLM Phase 2 evaluated 80 symbolic adversarial attack classes against the inherited Phase 1 formal models and included eight health and reachability checks. Under the stated models, symbolic cryptographic abstractions, attacker capabilities, and compromise conditions, all 80 tested attack classes were ruled out, no counterexample trace was

found, no attack class remained unresolved, and all eight health and reachability properties were reachable. The final A48 past-epoch identity property and its three helper lemmas were verified together in one integrated Tamarin invocation.

The official conceptual statement is:

> A different record, identifier, certificate, ledger history, or consensus outcome does not become the canonical lineage of an existing Child merely by being recorded or accepted as such. Canonical continuity requires the authenticated origin and succession relation defined by the OCLM model.

The official multi-blockchain statement is:

> OCLM is intended to operate as a higher lineage layer binding multiple blockchains and evidence systems. It does not replace their local consensus. It determines whether states accepted by different systems belong to the same authenticated canonical lineage.

11.10 Status at the End of Phase 2

At the completion of Phase 2:

```
Phase 1 formal foundation: completed
Phase 2 adversarial battery: completed
80 symbolic attack classes: resolved
8 health and reachability checks: reachable
A48 integrated proof: completed
Frozen final evidence: completed
Human-readable theory: completed
Multi-blockchain architectural position: defined
Assumptions and formal boundary: documented
```

The remaining work is not required to complete the Phase 2 result itself.

It belongs to subsequent phases, including:

- end-to-end refinement;
- implementation profiles;
- concrete cryptographic encodings;
- Oracle profiles;
- cross-chain finality profiles;
- privacy properties;
- recovery and governance;
- and production-system verification.

11.11 Final Perspective

OCLM does not claim that records are useless.

It claims that records are evidence, not origin itself.

It does not claim that consensus is meaningless.

It claims that consensus selects a state; it does not alone transform a different lineage into the original one.

It does not claim that systems must remain unchanged forever.

It distinguishes authorized continuity from substitution.

The resulting principle is:

```
Truth of origin is not created by record replacement.
Identity is not created by identifier reuse.
Continuity is not created by consensus alone.
Canonical lineage is preserved through authenticated generation and succession.
```

Phase 2 provides formal adversarial evidence for this principle within the stated OCLM models.

11.12 Closing Statement

The Oracle–Child Lineage Model defines identity through origin and continuity rather than through the continued dominance of a record.

Phase 2 establishes that the tested formalization resisted every one of its 80 defined symbolic adversarial classes while retaining reachable protocol behavior.

The completed result is therefore:

> A formally modeled and adversarially validated foundation for canonical lineage non-substitutability across independent records, epochs, authorities, and blockchain systems.

This concludes OCLM Phase 2.

References

[1] D. Dolev and A. C. Yao, "On the Security of Public Key Protocols," IEEE Transactions on Information Theory, vol. 29, no. 2, pp. 198–208, 1983. doi: 10.1109/TIT.1983.1056650.

[2] S. Meier, B. Schmidt, C. Cremers, and D. Basin, "The TAMARIN Prover for the Symbolic Analysis of Security Protocols," in Computer Aided Verification (CAV 2013), LNCS 8044, pp. 696–701, 2013. doi: 10.1007/978-3-642-39799-8_48.

[3] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.

[4] E. Buchman, J. Kwon, and Z. Milosevic, "The Latest Gossip on BFT Consensus," arXiv:1807.04938, 2018.

[5] B. Laurie, E. Messeri, and R. Stradling, "Certificate Transparency Version 2.0," RFC 9162, Internet Engineering Task Force, December 2021.

[6] World Wide Web Consortium, "Verifiable Credentials Data Model v2.0," W3C Recommendation, 15 May 2025.

[7] M. Herlihy, "Atomic Cross-Chain Swaps," in Proceedings of the 2018 ACM Symposium on Principles of Distributed Computing (PODC 2018), pp. 245–254, 2018. doi: 10.1145/3212734.3212736.

[8] A. Zamyatin, D. Harz, J. Lind, P. Panayiotou, A. Gervais, and W. J. Knottenbelt, "XCLAIM: Trustless, Interoperable, Cryptocurrency-Backed Assets," in 2019 IEEE Symposium on Security and Privacy, pp. 193–210, 2019. doi: 10.1109/SP.2019.00085.

[9] A. Zamyatin, M. Al-Bassam, D. Zindros, E. Kokoris-Kogias, P. Moreno-Sanchez, A. Kiayias, and W. J. Knottenbelt, "SoK: Communication Across Distributed Ledgers," in Financial Cryptography and Data Security, LNCS 12675, pp. 3–36, 2021. doi: 10.1007/978-3-662-64331-0_1.